

NLP

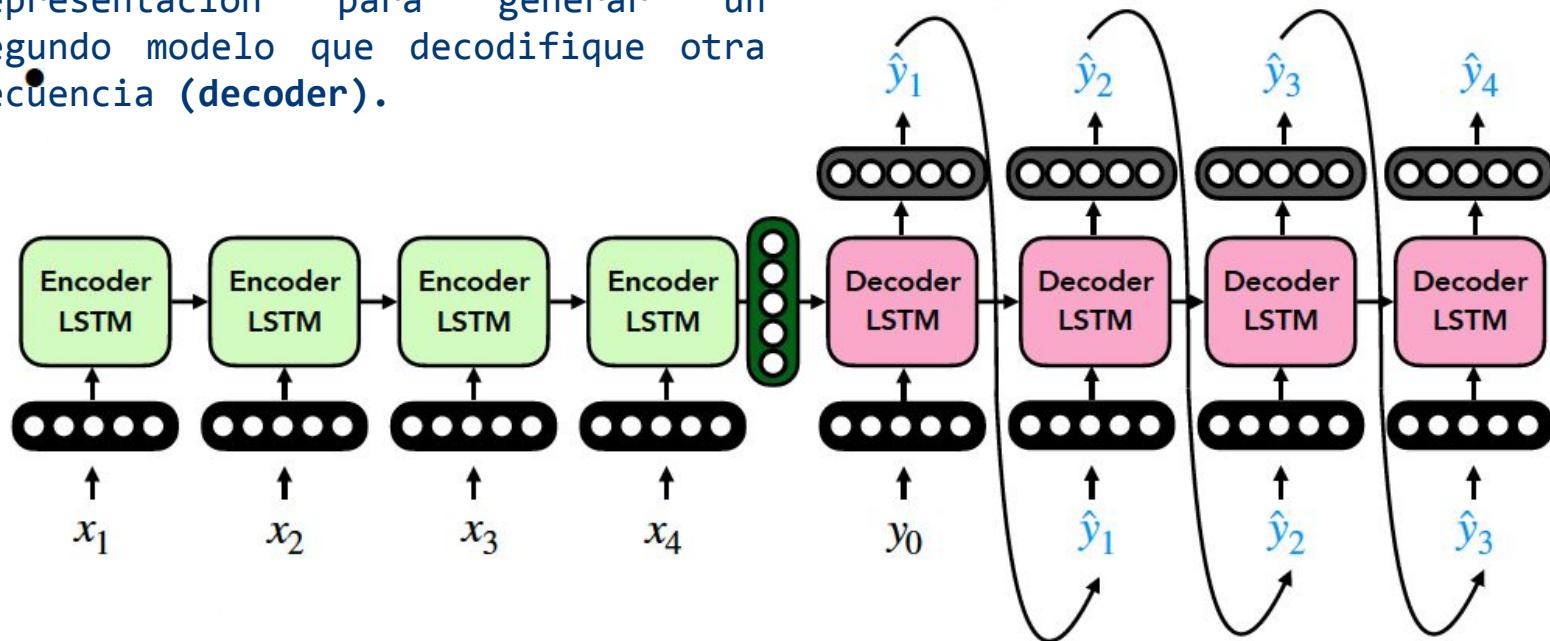
Celdas con Attention & Transformers

MSc. Fernando Silva
fernandosilva.clases@gmail.com

Seq2Seq



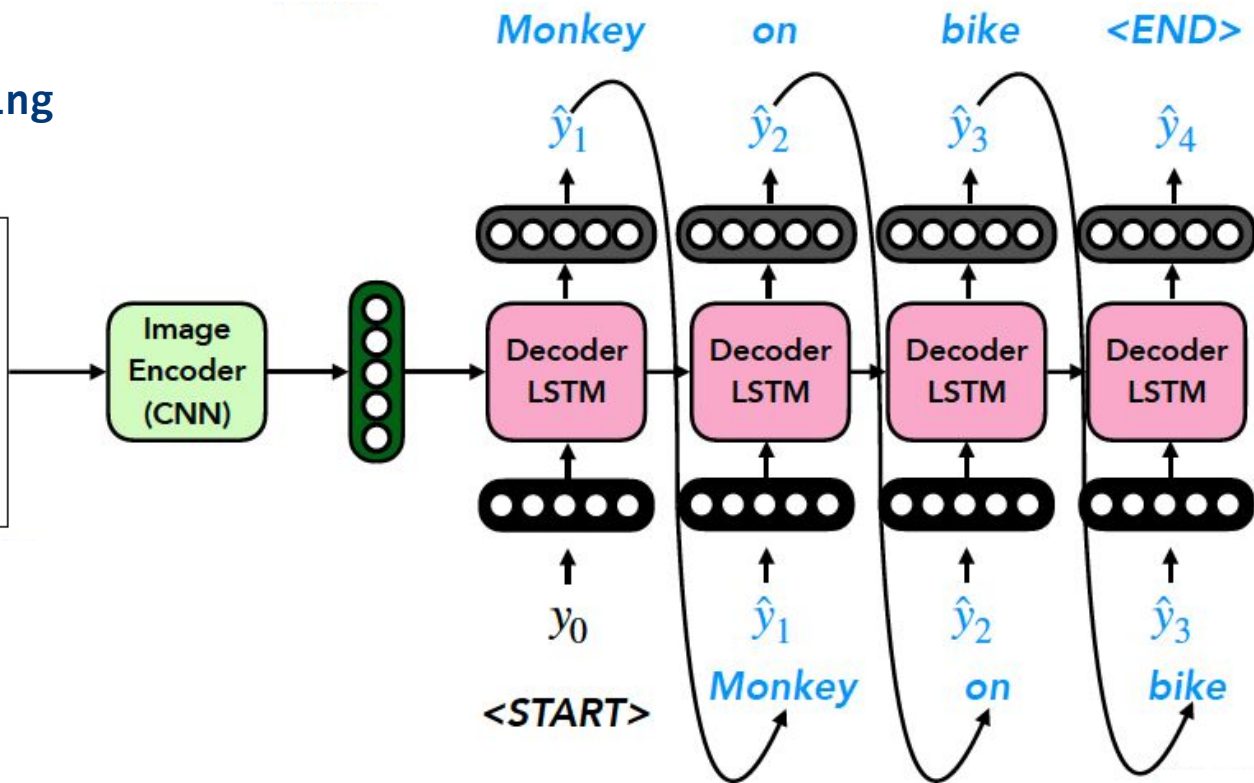
Codifica una secuencia completamente con un modelo (**encoder**) y utilizar su representación para generar un segundo modelo que decodifique otra secuencia (**decoder**).



Seq2Seq



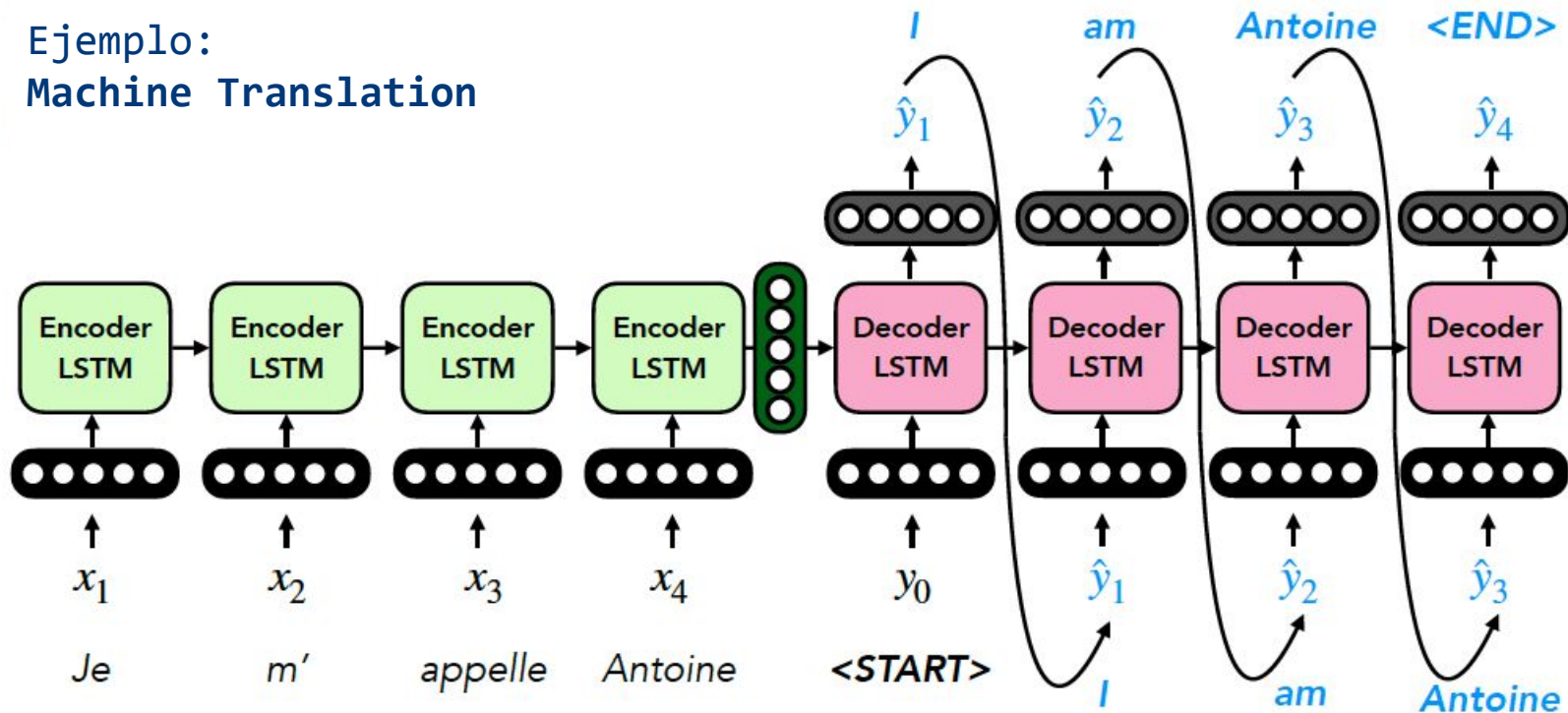
Ejemplo: Image Captioning



Seq2Seq



Ejemplo:
Machine Translation

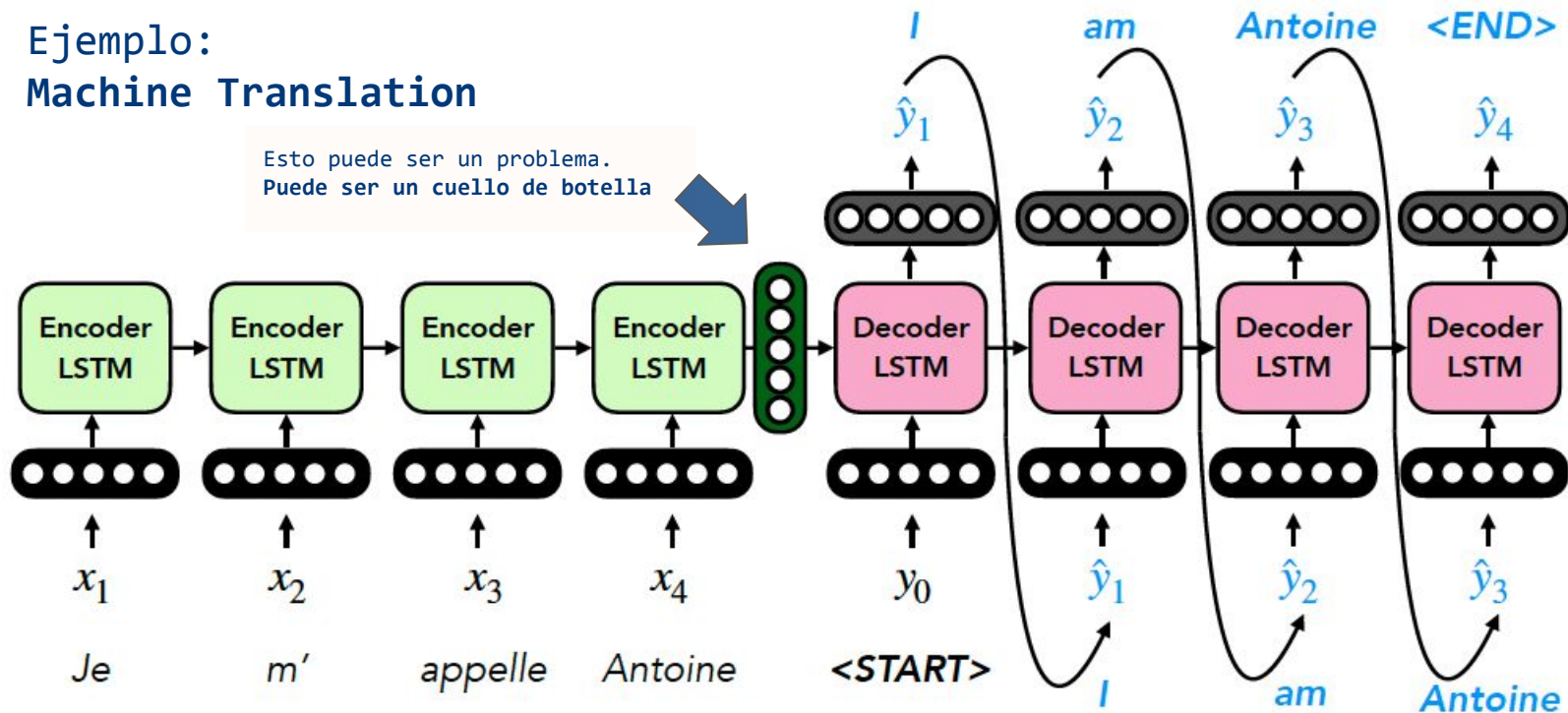


Seq2Seq



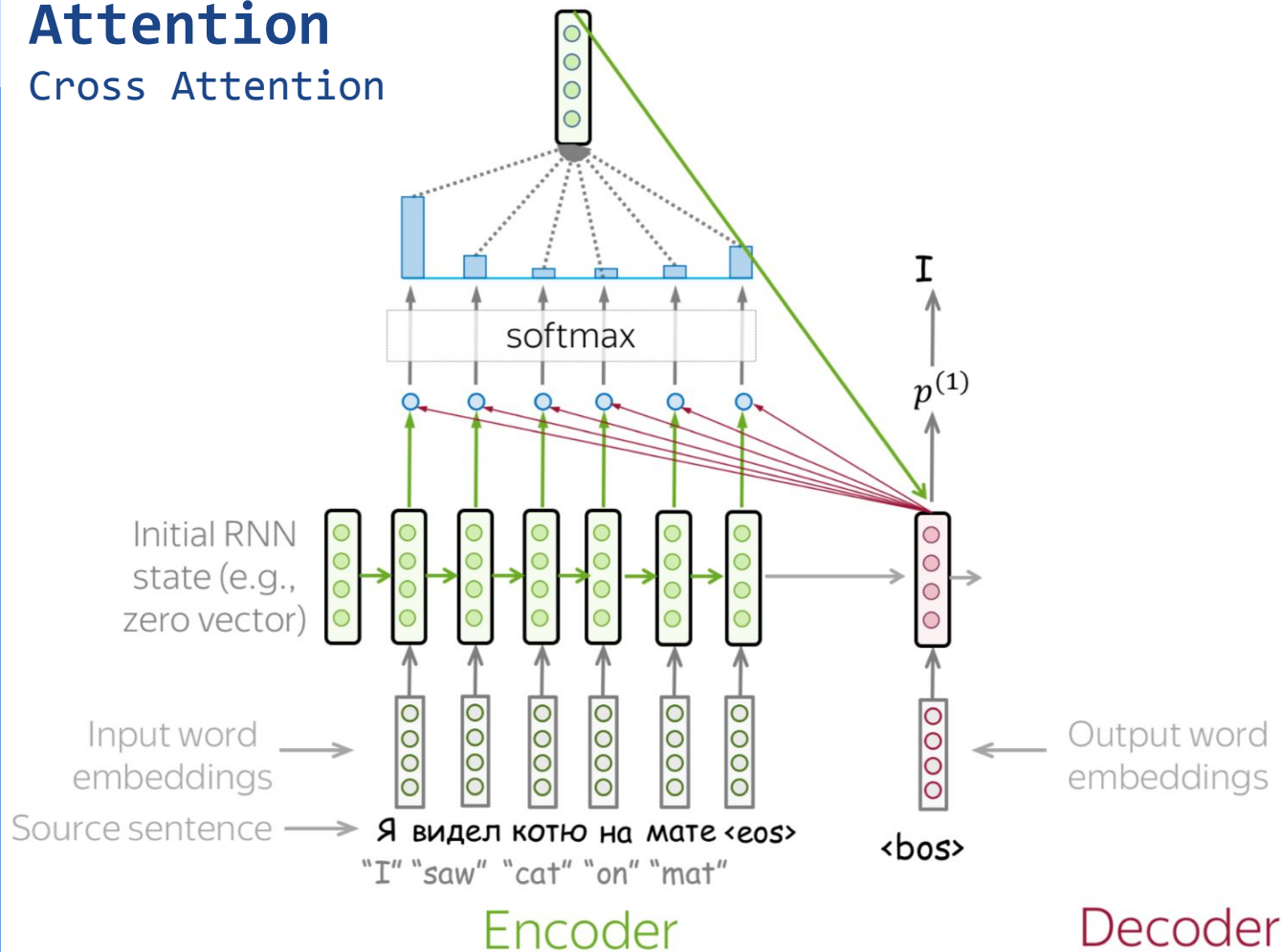
Ejemplo: Machine Translation

Esto puede ser un problema.
Puede ser un cuello de botella



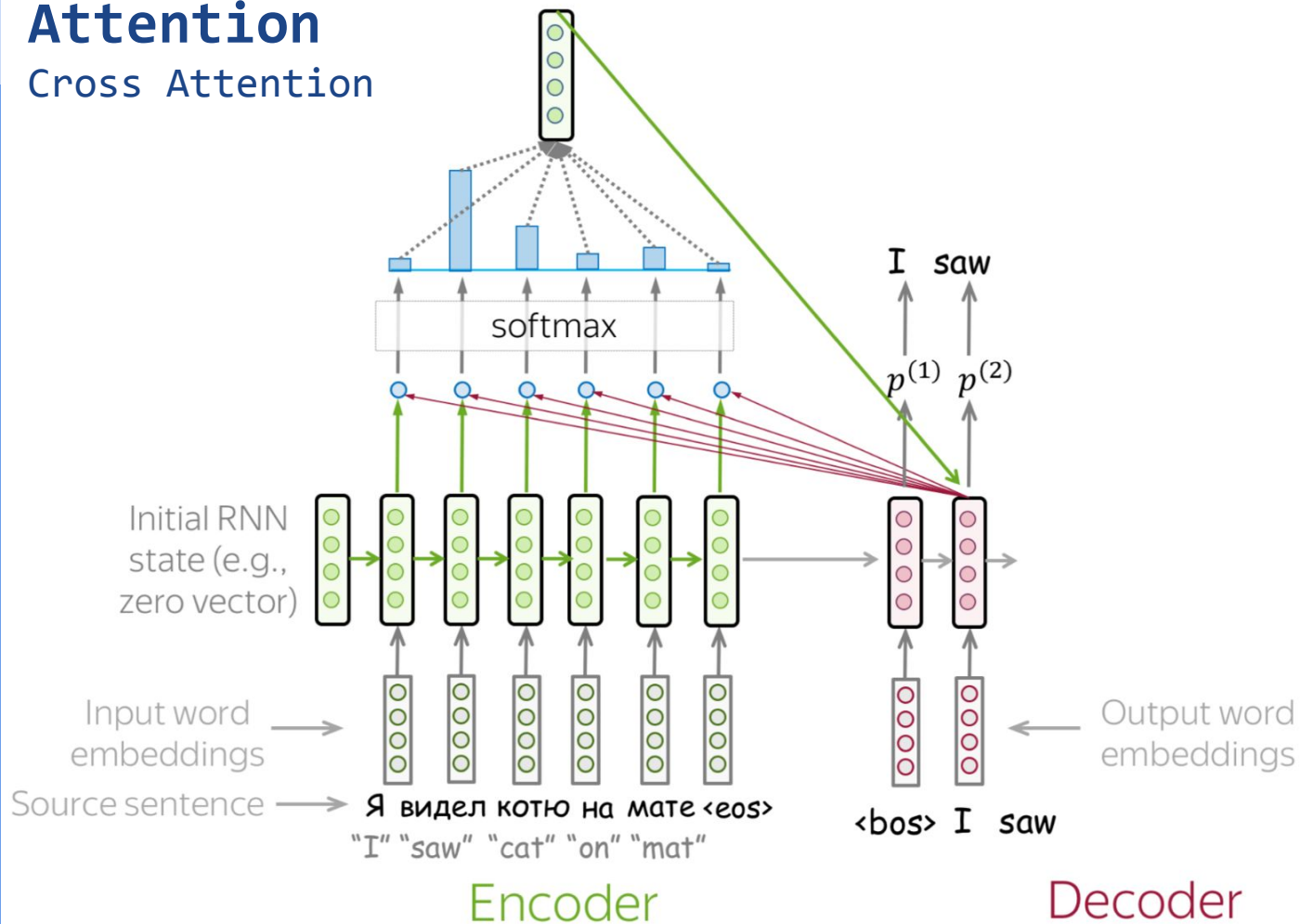
Attention

Cross Attention



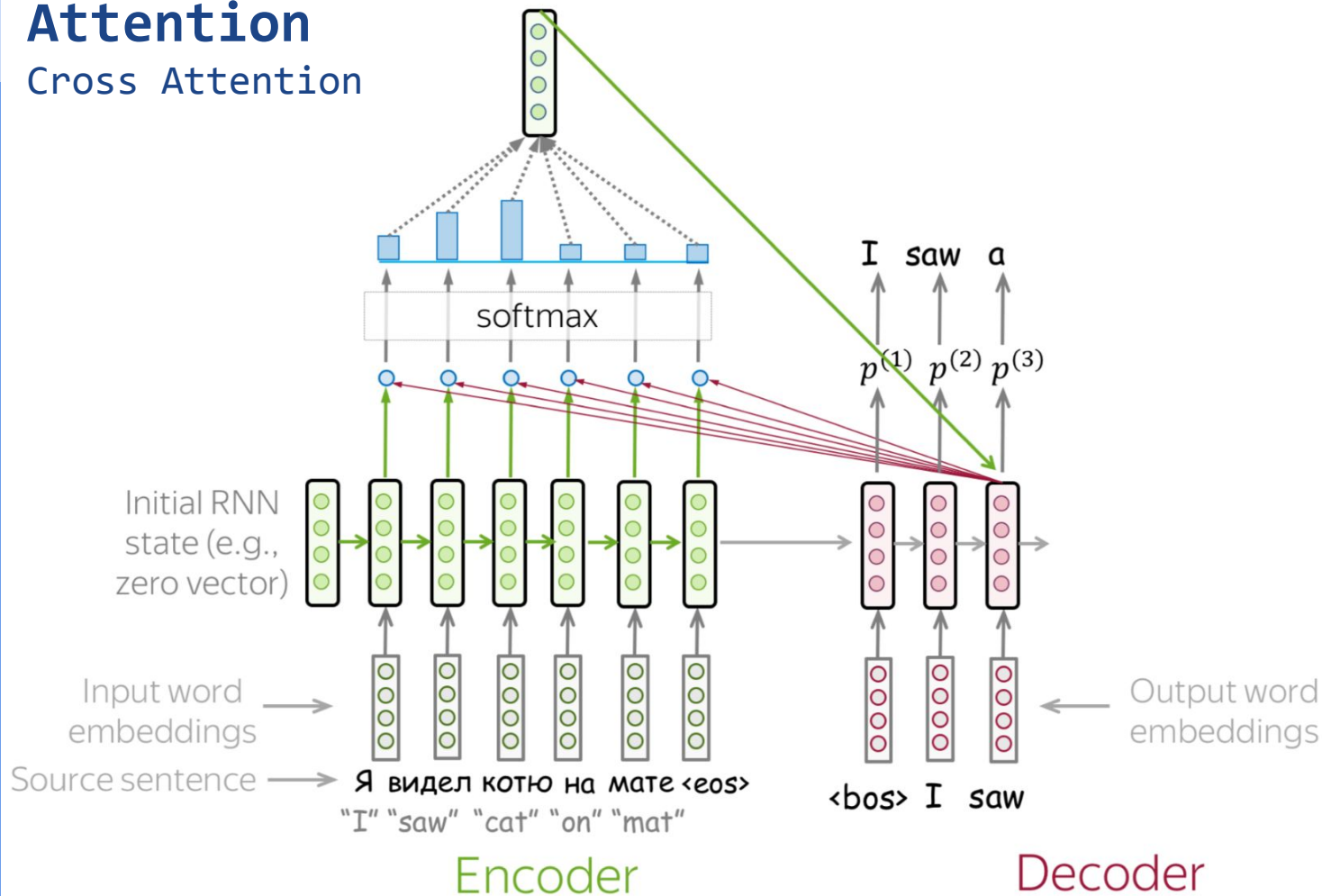
Attention

Cross Attention



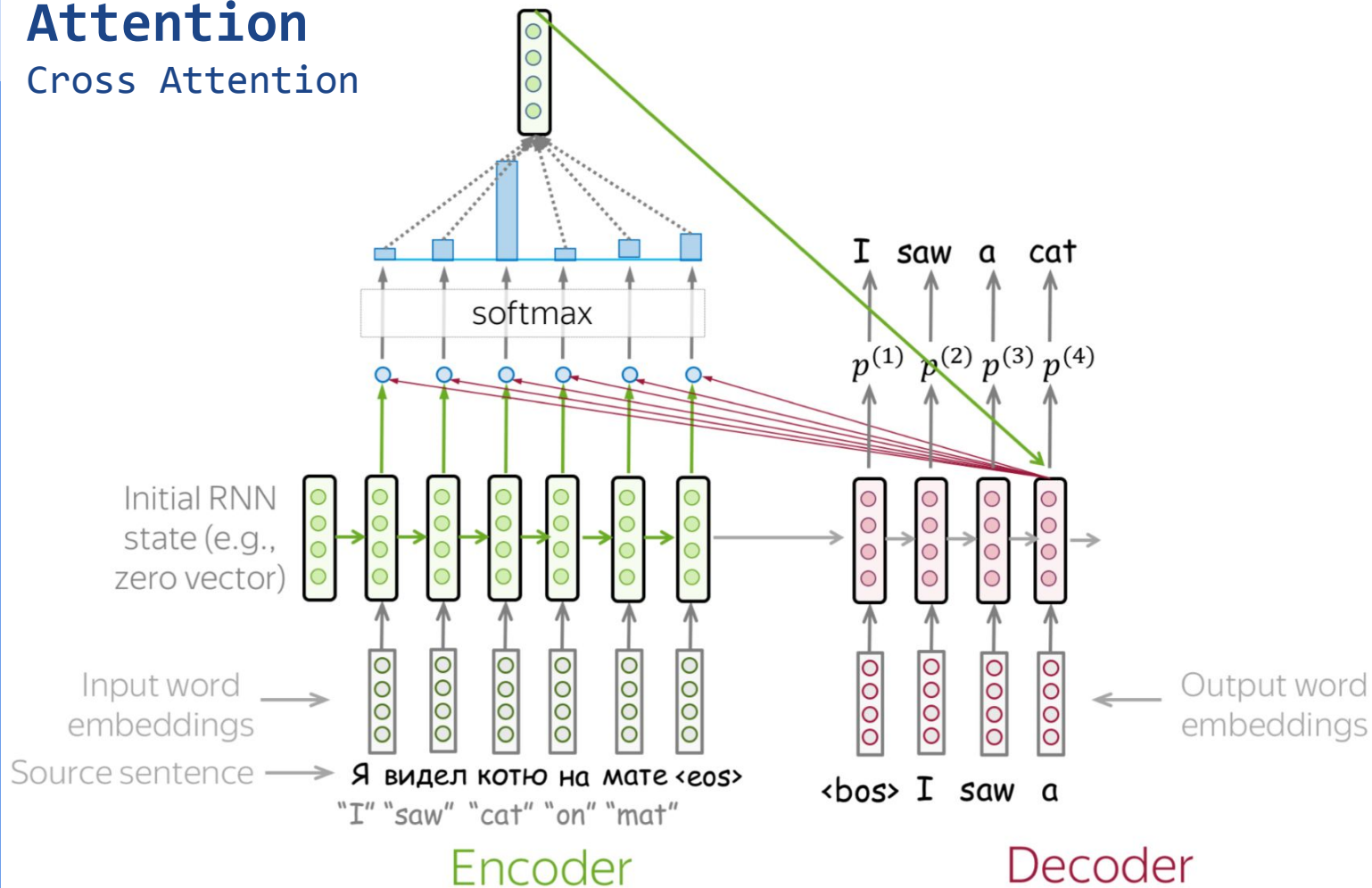
Attention

Cross Attention



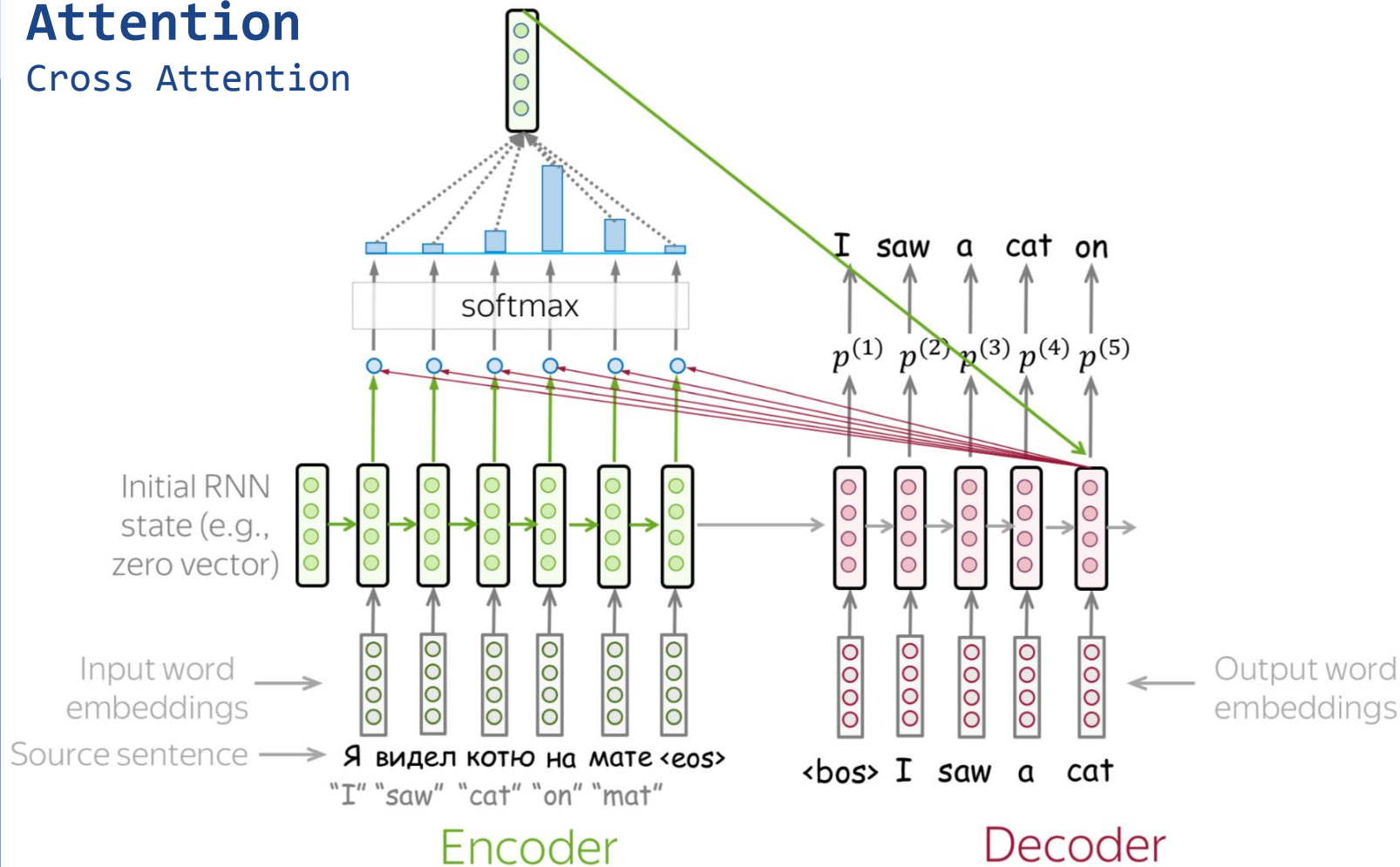
Attention

Cross Attention



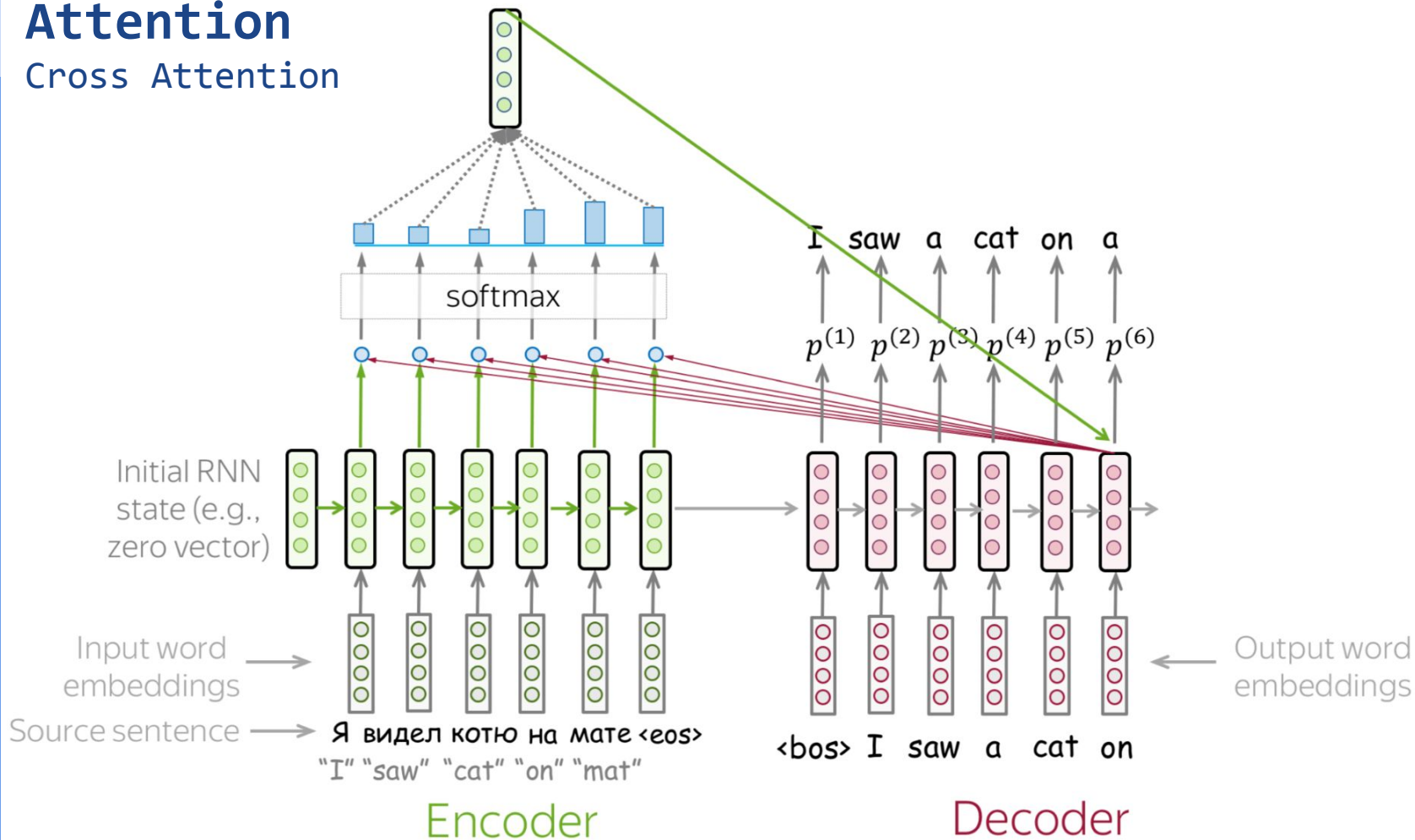
Attention

Cross Attention



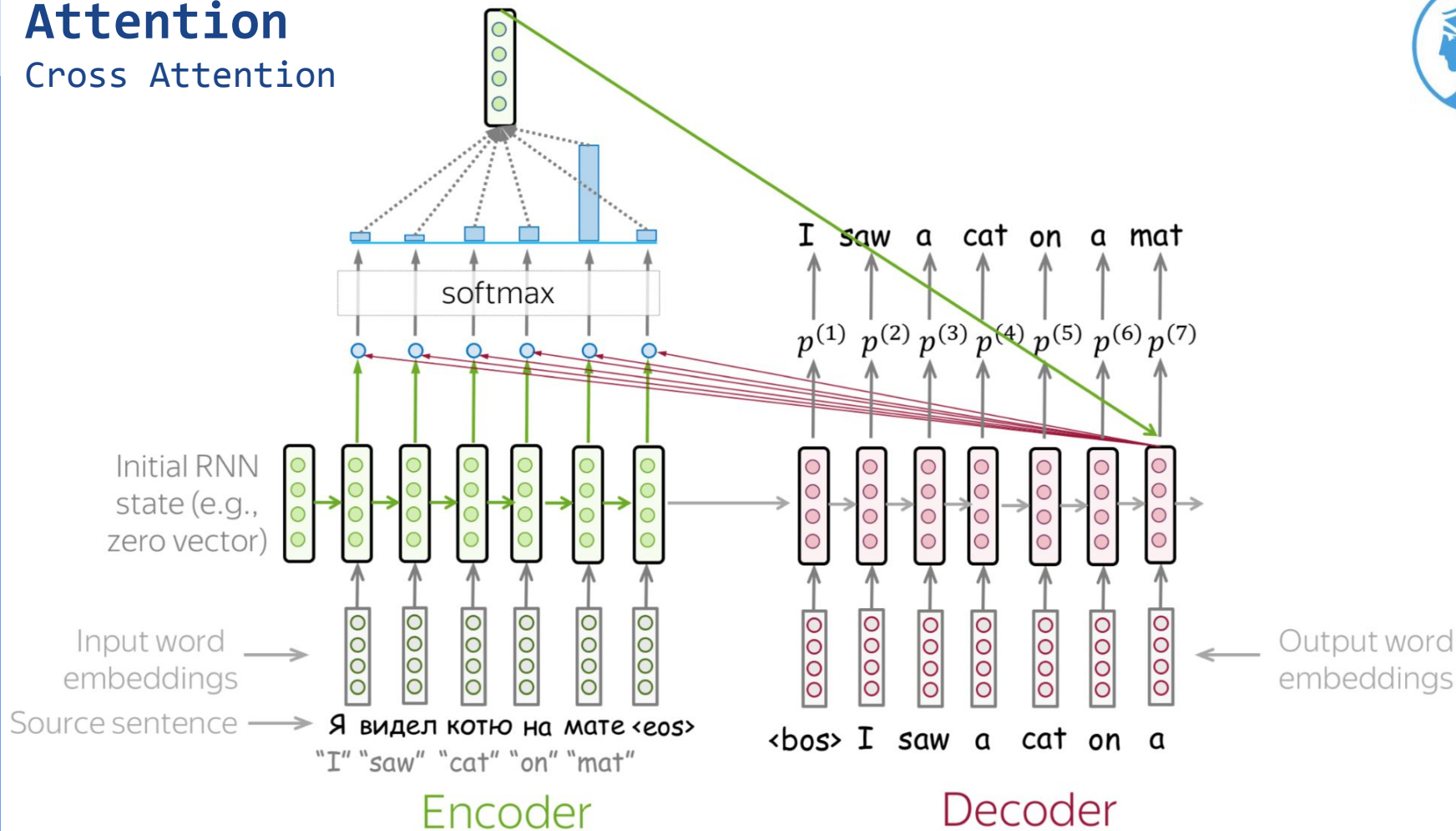
Attention

Cross Attention



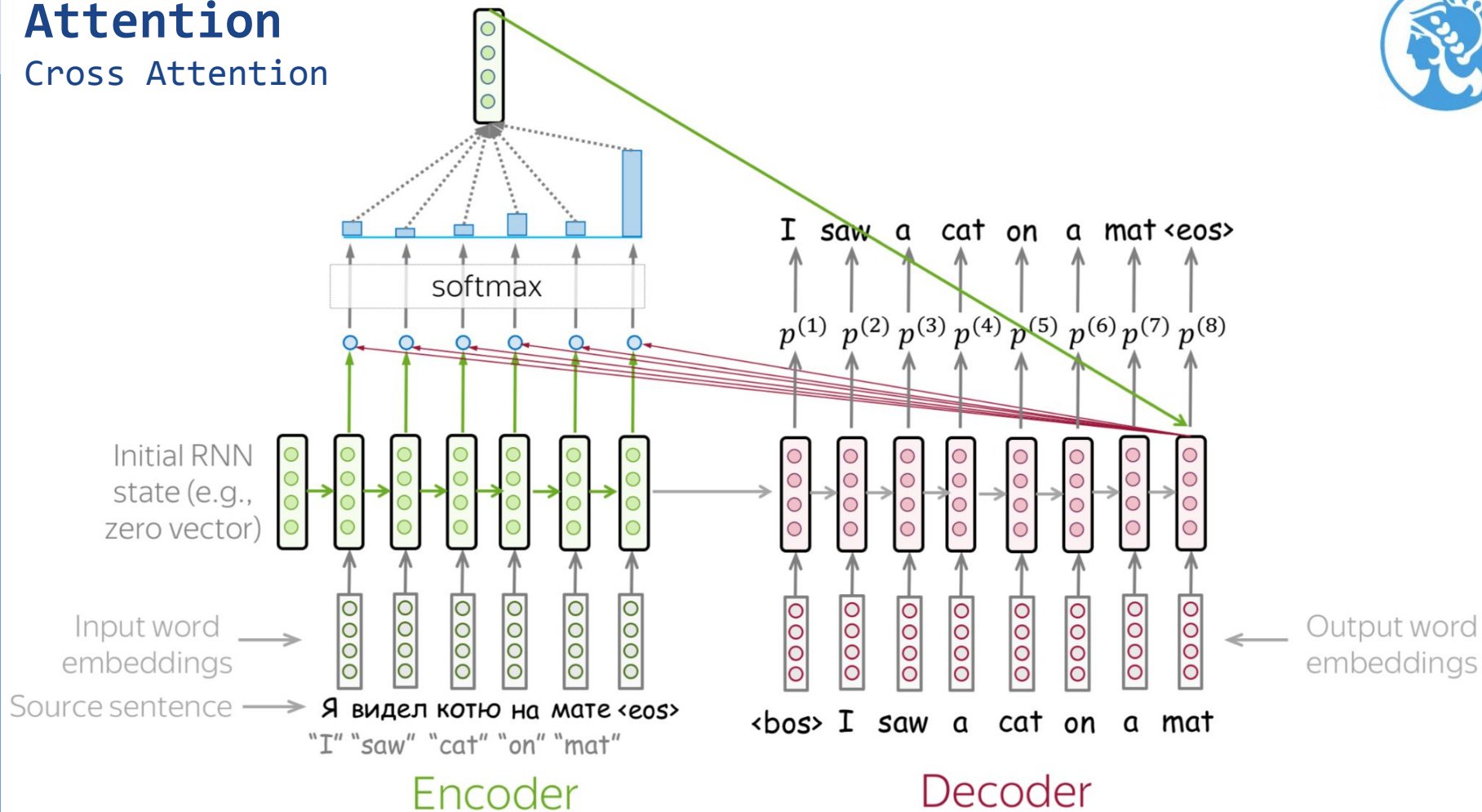
Attention

Cross Attention



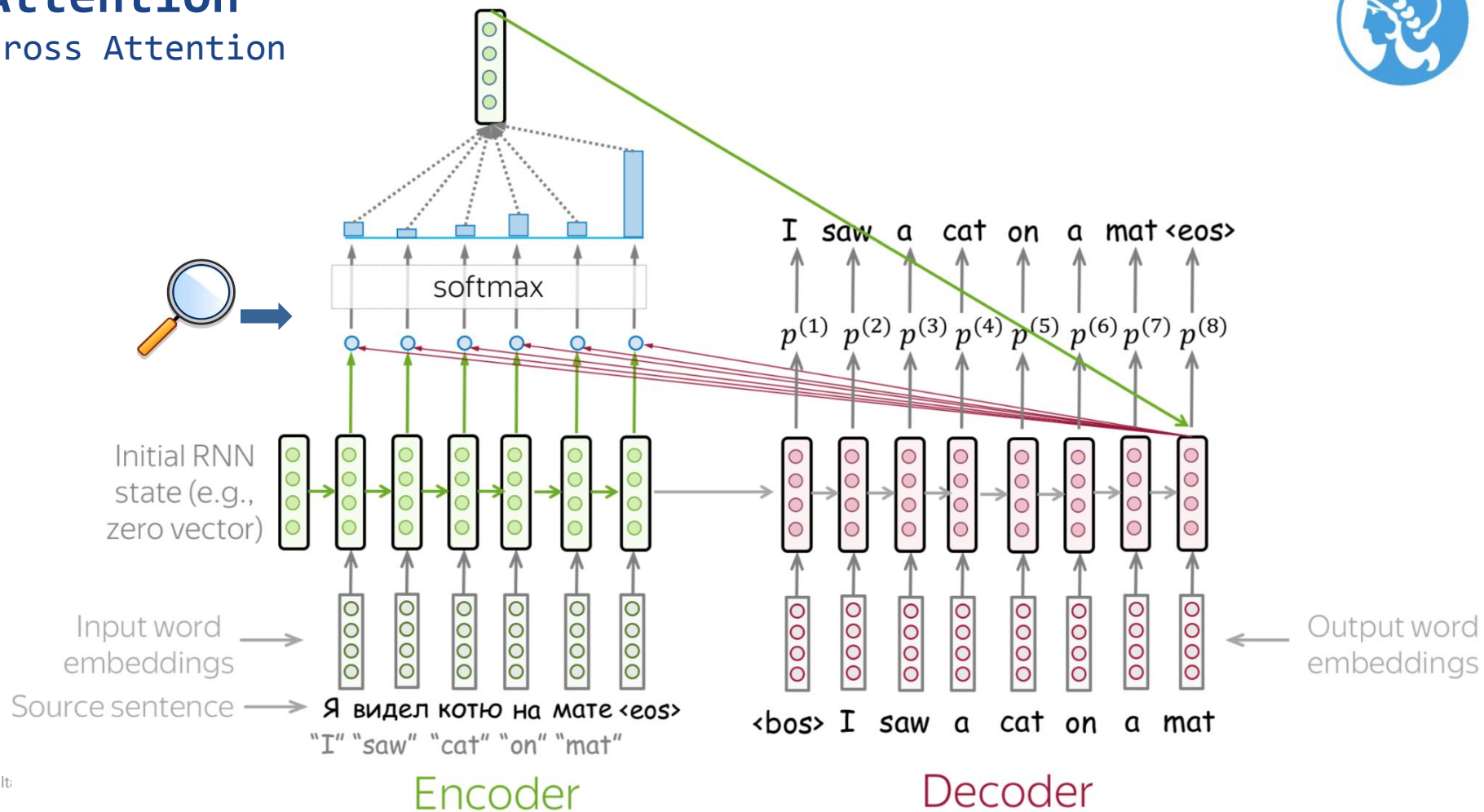
Attention

Cross Attention



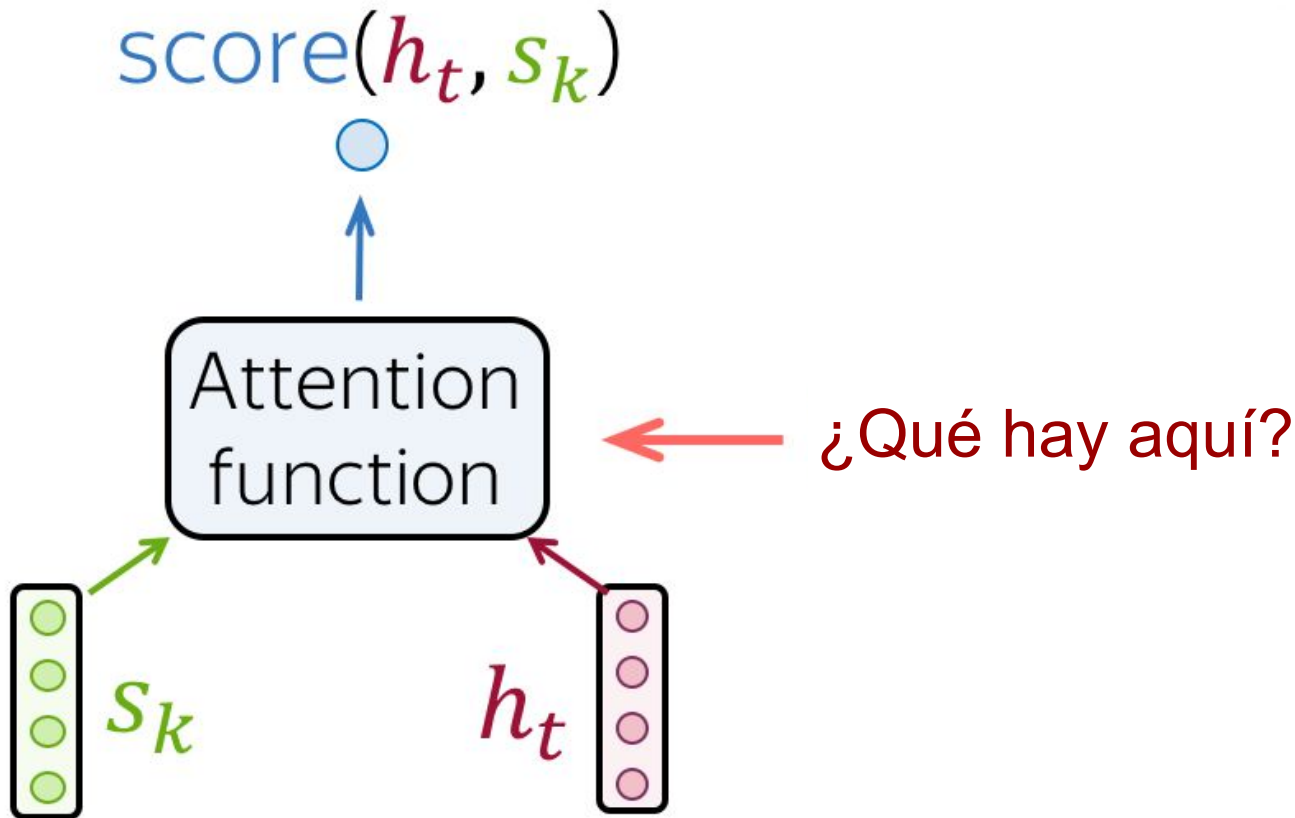
Attention

Cross Attention



Attention

Cross Attention



Attention

Cross Attention



Dot-product

$$\text{score}(h_t, s_k) = h_t^T s_k$$

Bilinear

$$\text{score}(h_t, s_k) = h_t^T W s_k$$

Multi-Layer Perceptron

$$\text{score}(h_t, s_k) = w_2^T \cdot \tanh(W_1 [h_t, s_k])$$

Attention

Cross Attention



Seq2seq without
attention

processing
within **encoder**

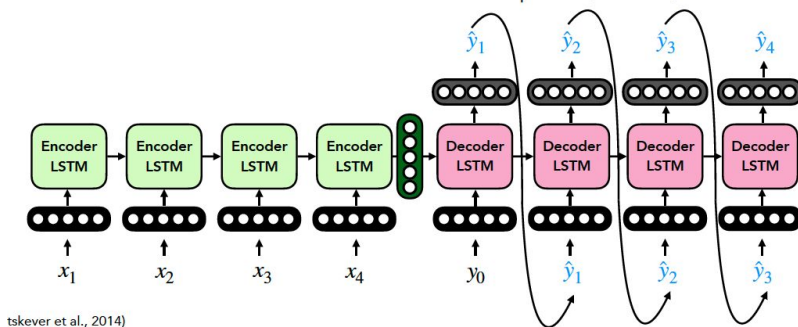
RNN/CNN

processing
within **decoder**

RNN/CNN

decoder-encoder
interaction

static fixed-
sized vector



tskever et al., 2014)

Attention

Cross Attention



Seq2seq without
attention

processing
within **encoder**

RNN/CNN

processing
within **decoder**

RNN/CNN

decoder-encoder
interaction

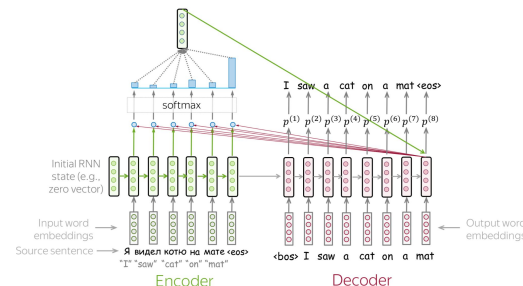
static fixed-
sized vector

Seq2seq with
attention

RNN/CNN

RNN/CNN

attention



Attention

Cross Attention



Seq2seq without attention

processing
within **encoder**

RNN/CNN

processing
within **decoder**

RNN/CNN

decoder-encoder
interaction

static fixed-
sized vector

Seq2seq with attention

RNN/CNN

RNN/CNN

attention

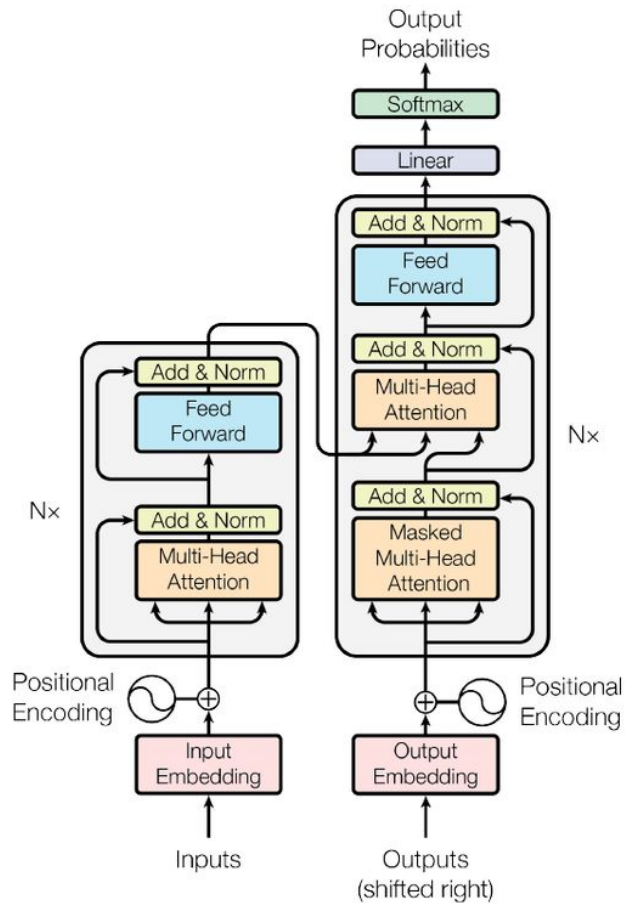
Transformer

attention

attention

attention

Transformer

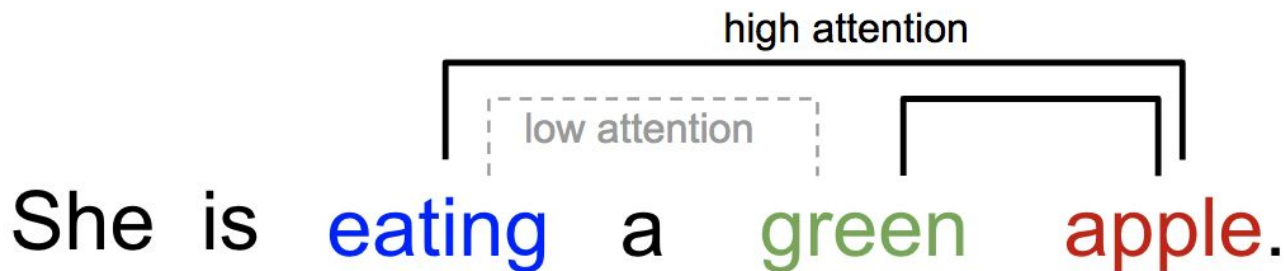


Transformer

Self-Attention



- ❖ Self-attention es el mecanismo mediante el cual cada token de una secuencia interactúa con los demás tokens.
- ❖ Cada token “atiende” a otros tokens de la oración, asigna distintos niveles de importancia a cada uno, y utiliza esa información para construir una nueva representación contextualizada de sí mismo.



Transformer

Self-Attention



En self-attention, cada token se proyecta en tres representaciones distintas:

- ❖ Query (Q): representa lo que el token está buscando.
- ❖ Key (K): representa qué tipo de información ofrece ese token.
- ❖ Value (V): contiene la información que será utilizada para construir la nueva representación.

La similitud entre Query y Key determina qué tan relevante es un token para otro, y esos pesos se usan para combinar los Values.



Transformer

Self-Attention



Each vector receives three representations (“roles”)

$$\begin{bmatrix} W_Q \end{bmatrix} \times \begin{bmatrix} \text{green} \\ \text{green} \\ \text{green} \end{bmatrix} = \begin{bmatrix} \text{blue} \\ \text{blue} \\ \text{blue} \end{bmatrix}$$

Query: vector **from** which the attention is looking

“Hey there, do you have this information?”

$$\begin{bmatrix} W_K \end{bmatrix} \times \begin{bmatrix} \text{green} \\ \text{green} \\ \text{green} \end{bmatrix} = \begin{bmatrix} \text{yellow} \\ \text{yellow} \\ \text{yellow} \end{bmatrix}$$

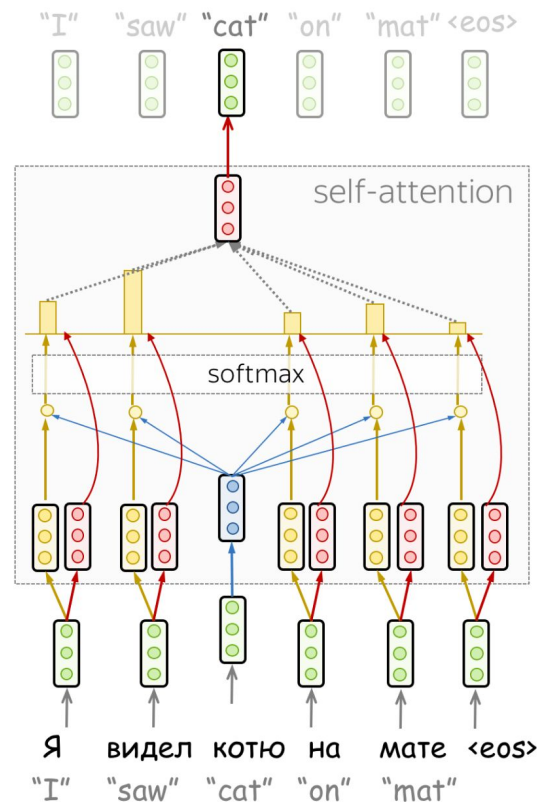
Key: vector **at** which the query looks to compute weights

“Hi, I have this information – give me a large weight!”

$$\begin{bmatrix} W_V \end{bmatrix} \times \begin{bmatrix} \text{green} \\ \text{green} \\ \text{green} \end{bmatrix} = \begin{bmatrix} \text{red} \\ \text{red} \\ \text{red} \end{bmatrix}$$

Value: their weighted sum is attention output

“Here’s the information I have!”



Transformer

Self-Attention



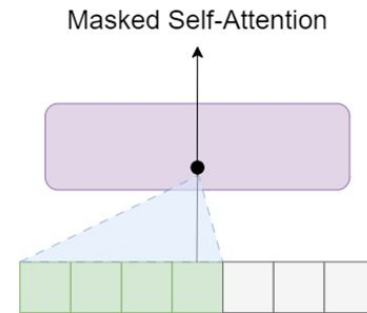
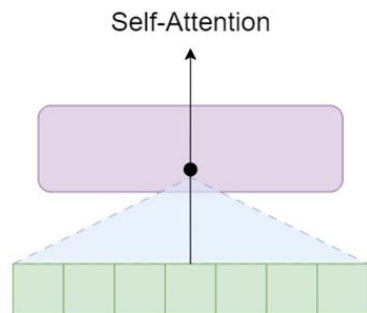
$$\textit{Attention}(Q, K, V) = \textit{SoftMax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- ❖ La dimensión de los vectores Key (K) (y también de los Query (Q))
- ❖ Se usa para escalar el producto QK^T y evitar valores demasiado grandes.

Transformer

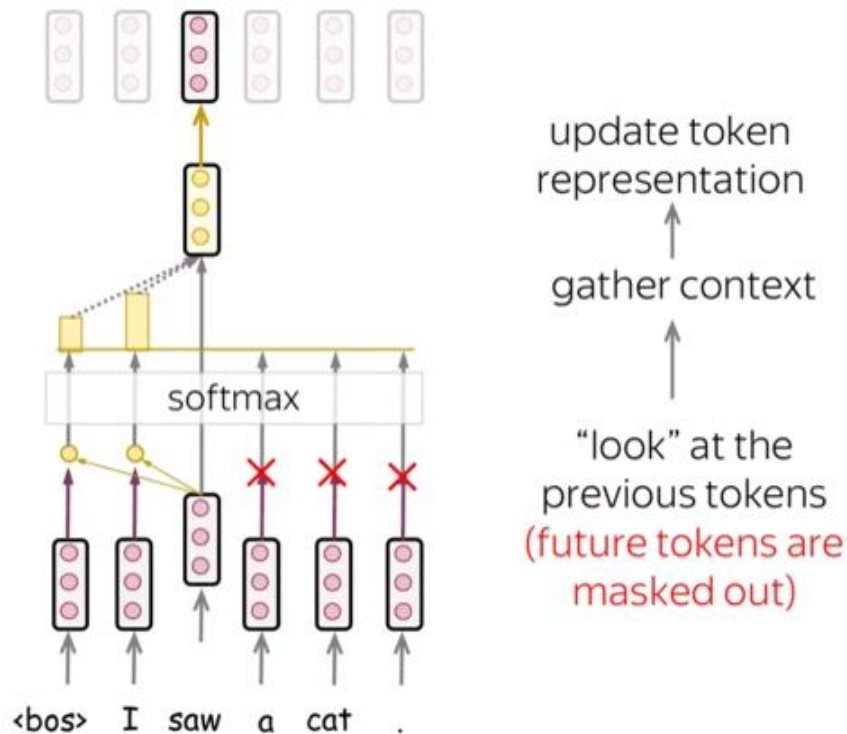
Masked Self-Attention

- ❖ El **encoder** recibe todos los tokens al mismo tiempo, por lo que cada token puede atender a cualquier otro token de la secuencia mediante **self-attention**.
- ❖ En cambio, el **decoder** genera la secuencia token por token. En cada paso, solo tiene acceso a los tokens generados hasta ese momento.
- ❖ Para evitar que el modelo “vea el futuro”, se utiliza **masked self-attention**, donde las posiciones futuras se enmascaran, impidiendo que influyan en la predicción actual.

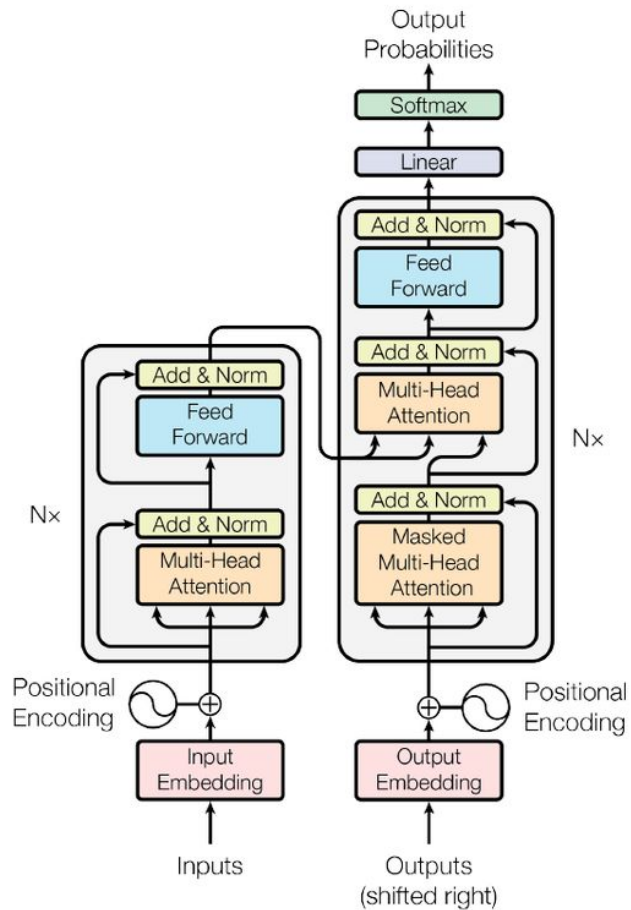


Transformer

Masked Self-Attention



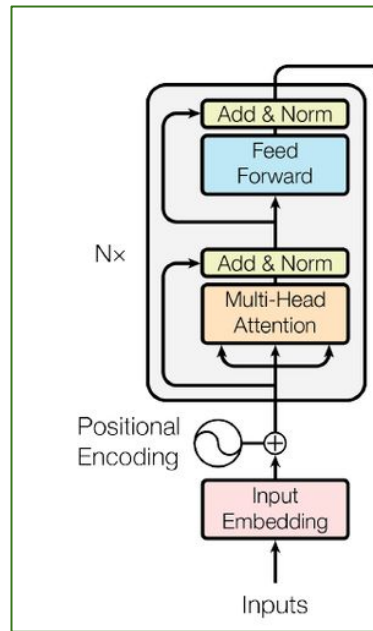
Transformer



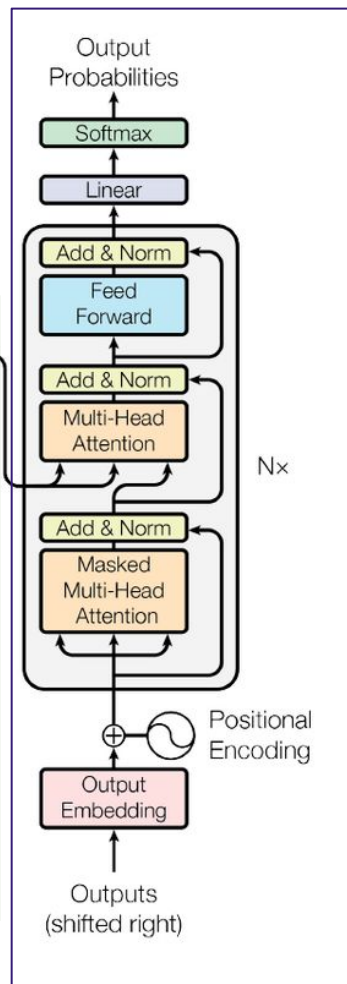
Transformer

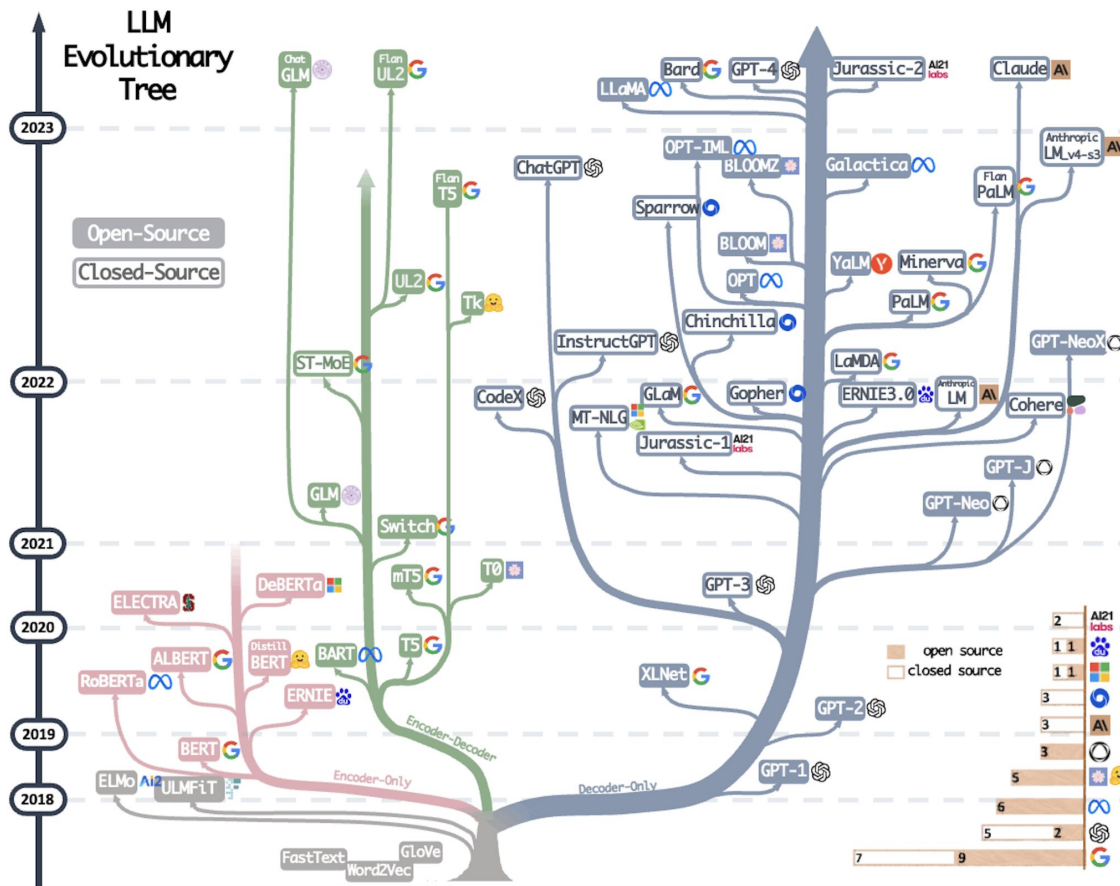


Encoder Representación

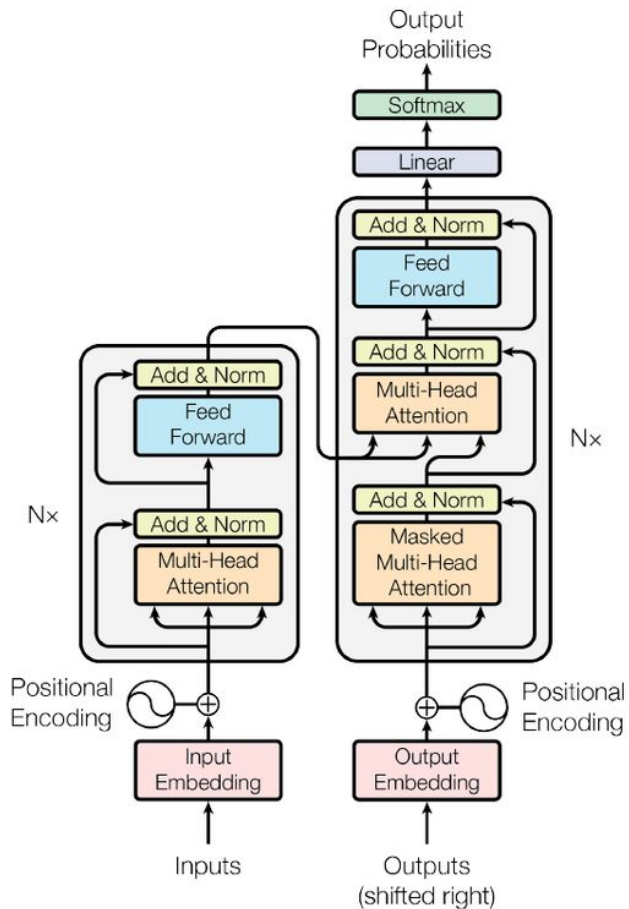


Decoder Generación





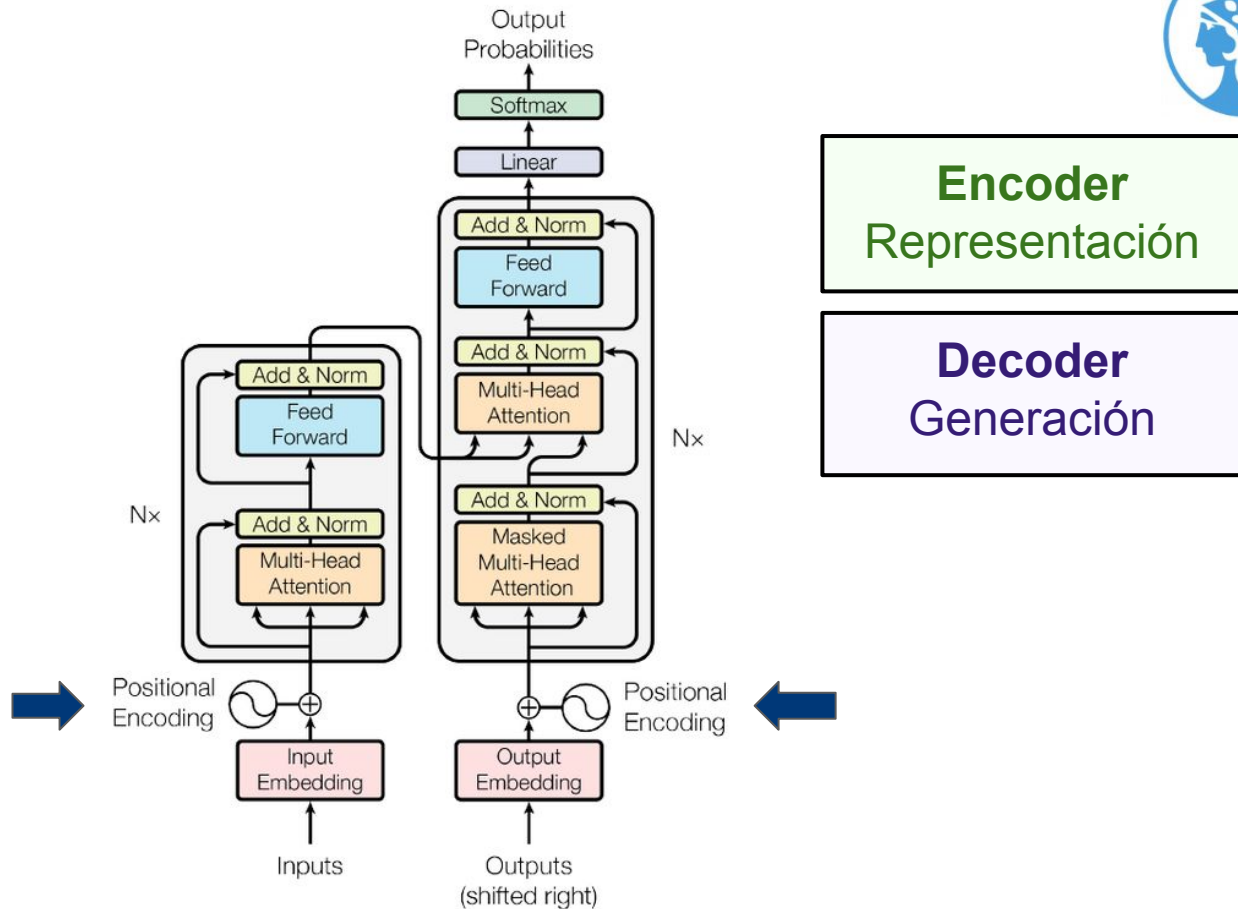
Transformer



Encoder
Representación

Decoder
Generación

Transformer

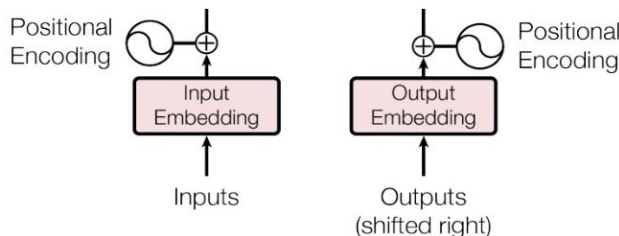


Transformer

Positional Encoding



- ❖ El mecanismo de self-attention no tiene información sobre el orden de las palabras por sí solo.
- ❖ A diferencia de modelos como RNN o LSTM, que procesan las secuencias paso a paso, el self-attention procesa todos los tokens en paralelo.
- ❖ Esto significa que, sin información adicional, el modelo no puede distinguir la posición de cada palabra ni saber cuál viene antes o después.



- La idea es **añadir un vector adicional a cada embedding** que representa la posición de la palabra en la secuencia.

Transformer

Positional Encoding



Ejemplo:

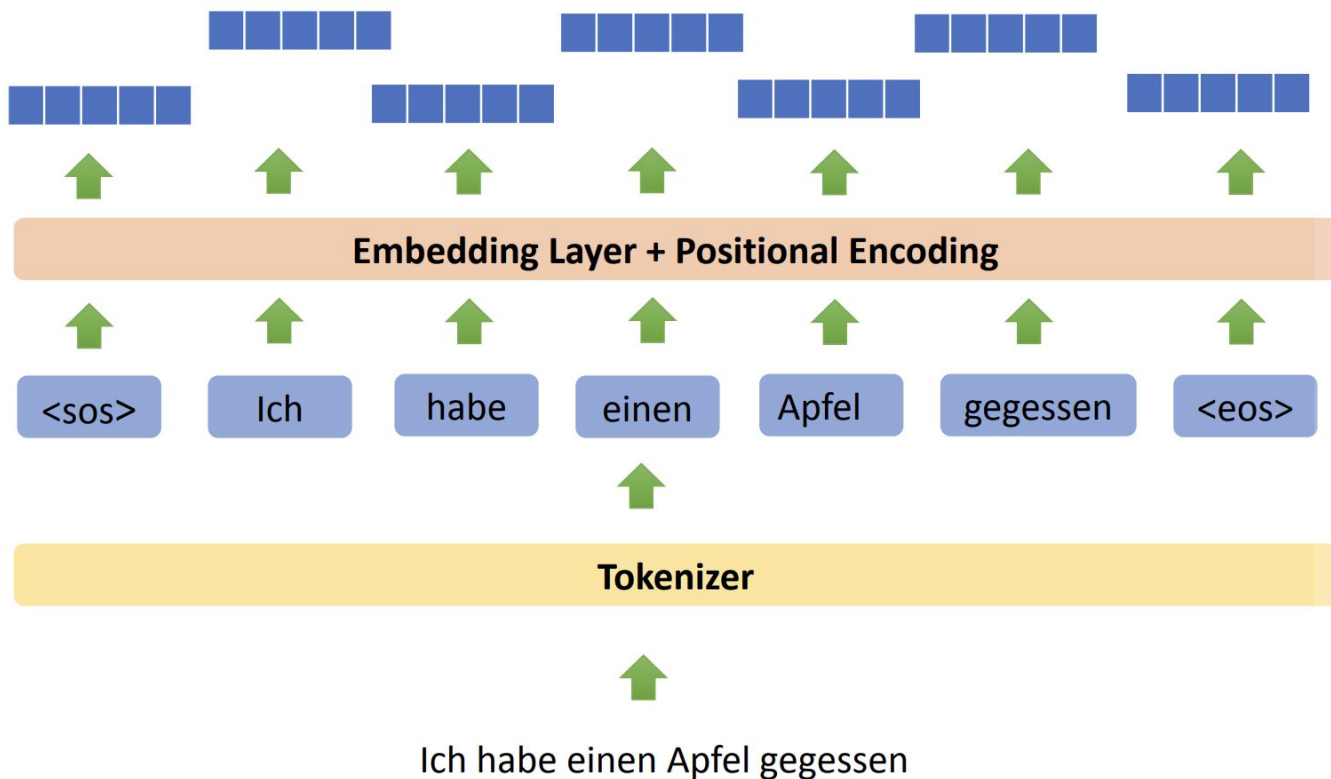
- ❖ "dog" en la posición 1 $\rightarrow$ `embedding(dog) + encoding(pos=1)`
- ❖ "dog" en la posición 5 $\rightarrow$ `embedding(dog) + encoding(pos=5)`

Formas de representar la posición:

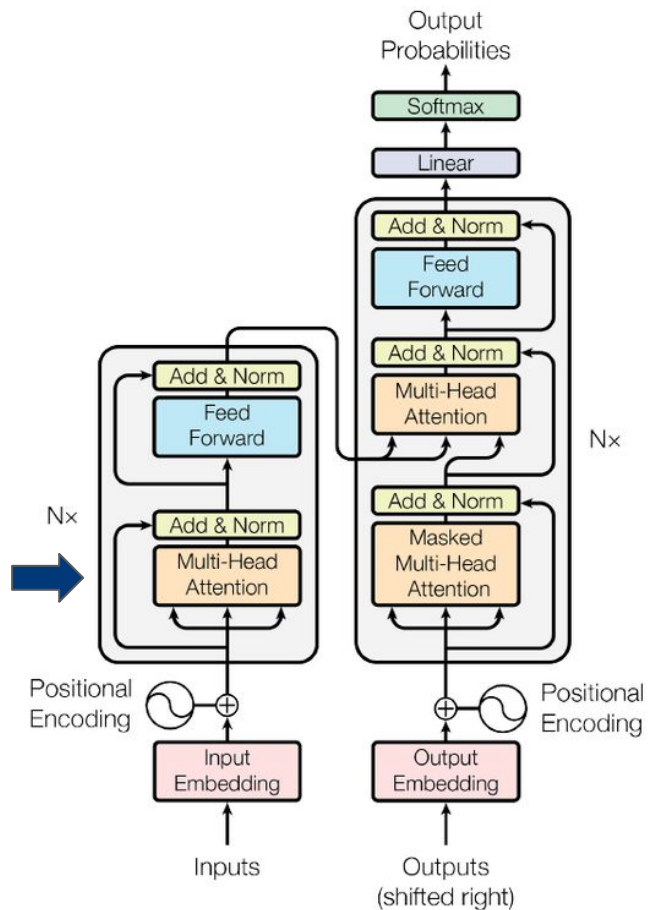
- **Predefinida (función matemática):** Los transformers originales (Vaswani et al., 2017) usan una **función sinusoidal**. Esto hace que los vectores de posiciones cercanas estén relacionados (capturan distancias).
- **Aprendida (trainable):** Algunos modelos aprenden directamente los vectores de posición como parámetros, igual que los embeddings de palabras.

Transformer

Positional Encoding



Transformer

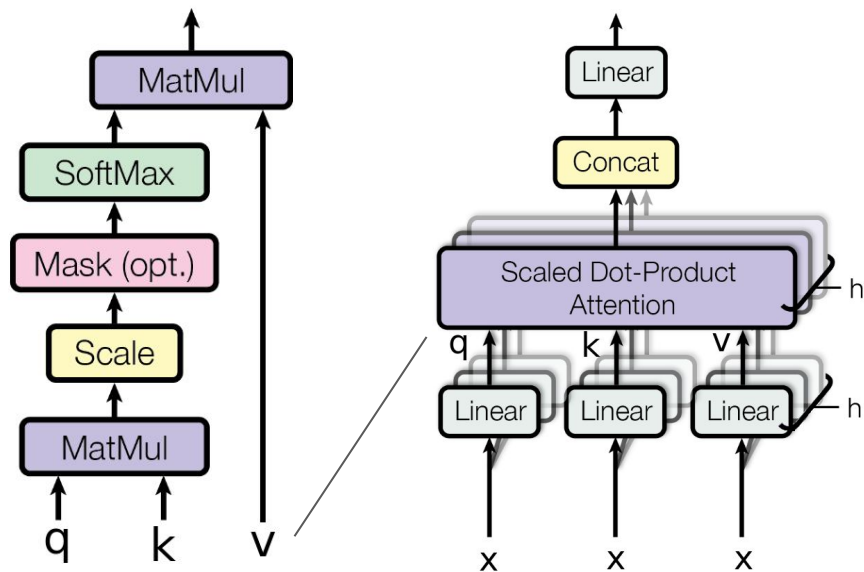


Encoder
Representación

Decoder
Generación

Transformer

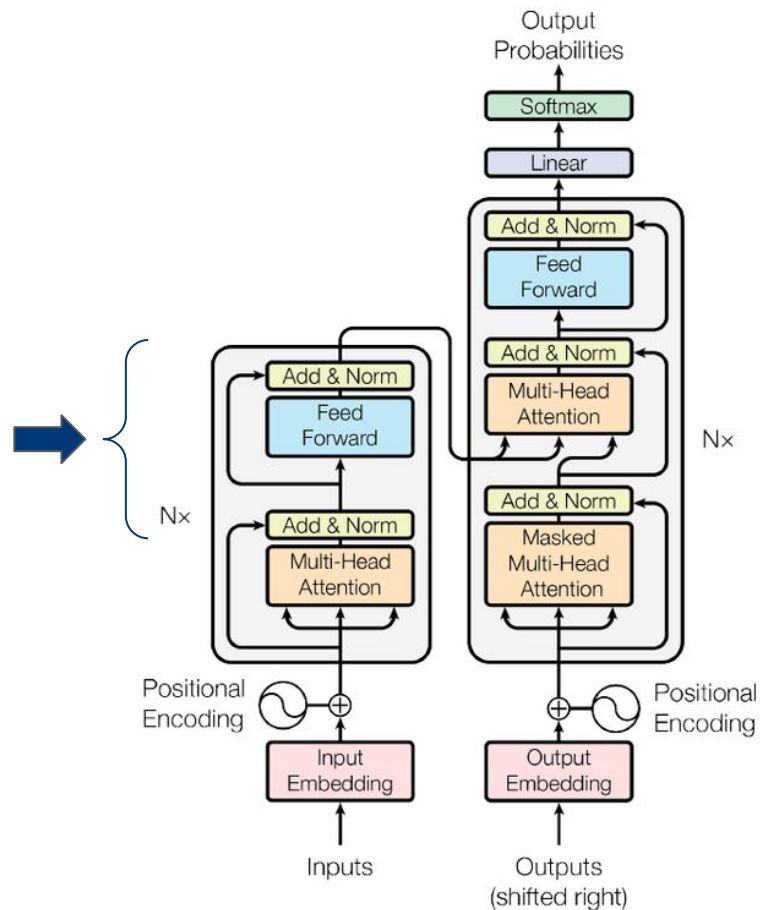
Multi-Head Attention



Multi-head es similar a tener varios filtros convolucionales en redes convolucionales.

Cada módulo de atención puede aprender a “atender” diferentes relaciones entre los elementos de la secuencia.

Transformer

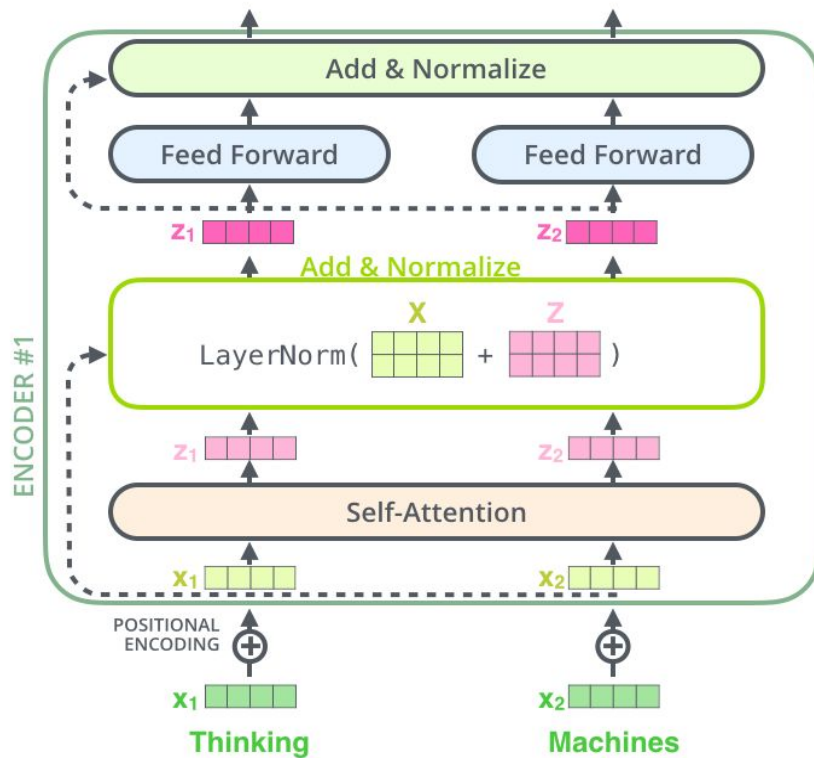


Encoder
Representación

Decoder
Generación

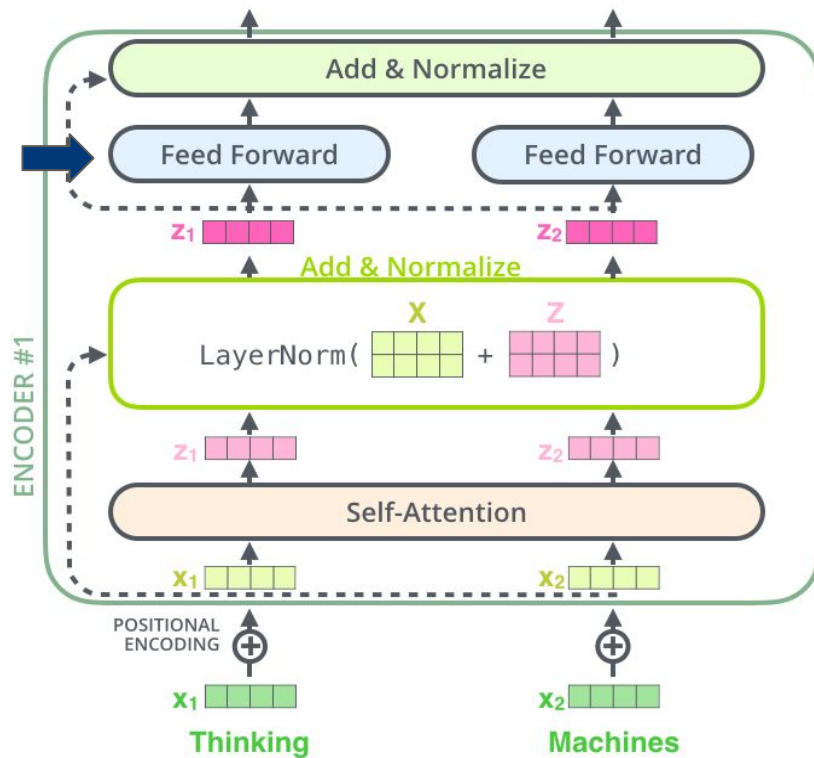
Transformer

Add & Norm



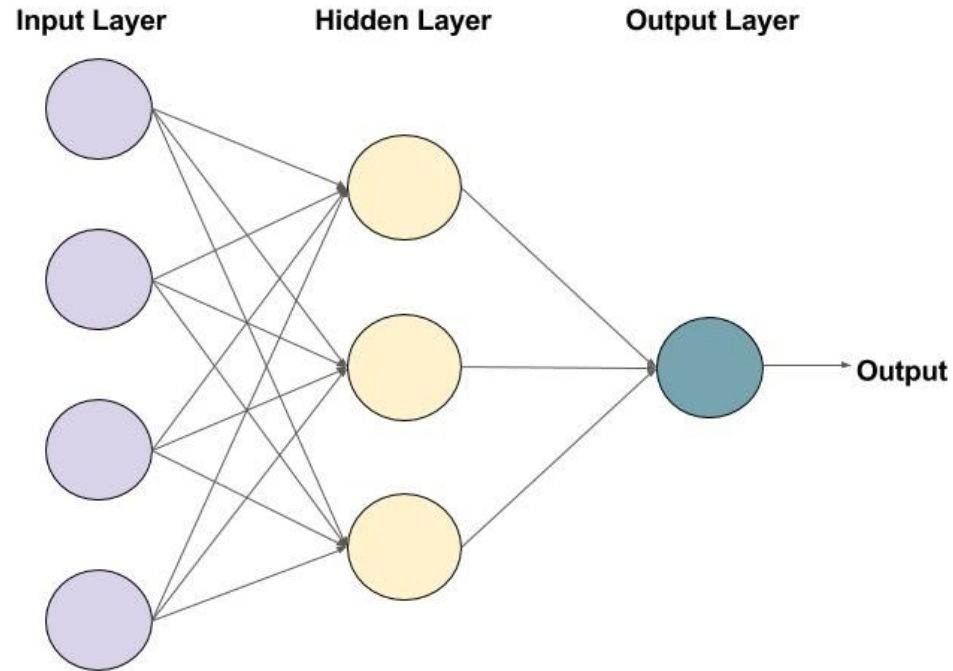
Transformer

Add & Norm

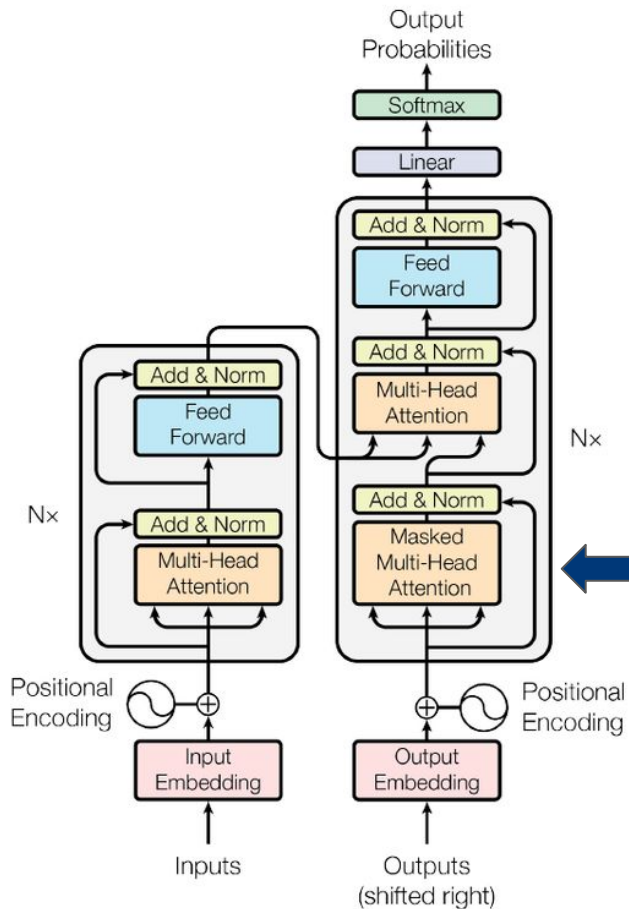


Transformer

Feed Forward



Transformer

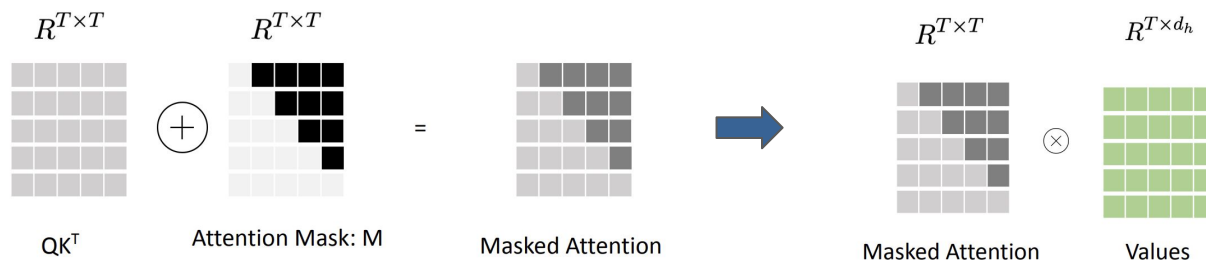


Encoder
Representación

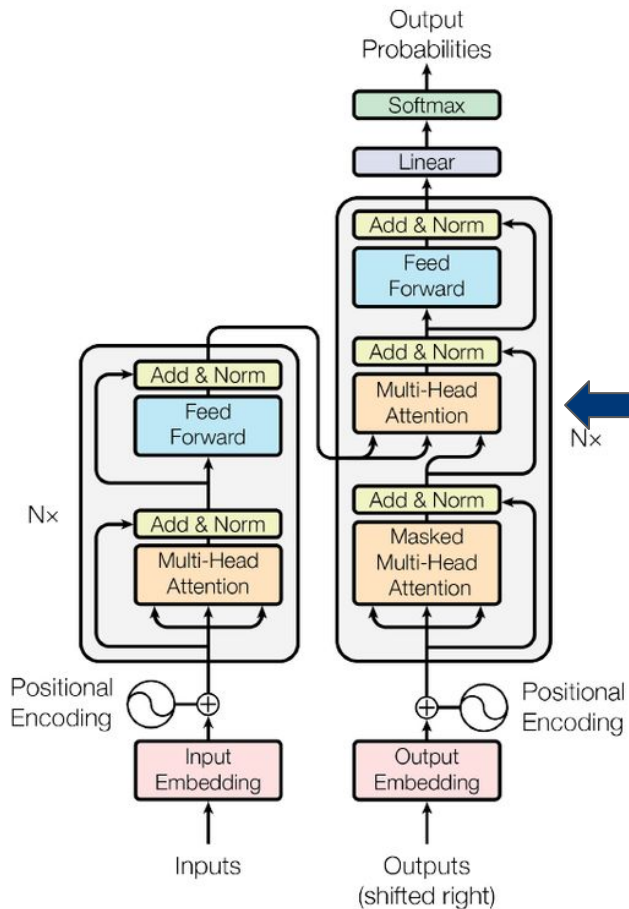
Decoder
Generación

Transformer

Masked Multi-Head Attention



Transformer



Encoder
Representación

Decoder
Generación

Transformer

Encoder - Decoder



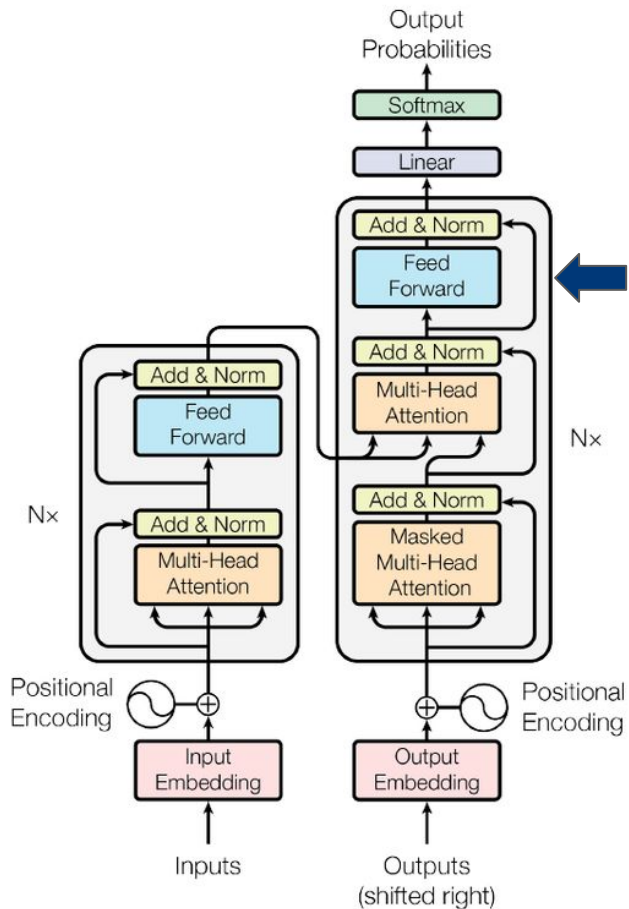
Encoder

Keys from **Encoder Outputs**
Values from **Encoder Outputs**

Decoder

Queries from **Decoder Inputs**

Transformer



Encoder
Representación

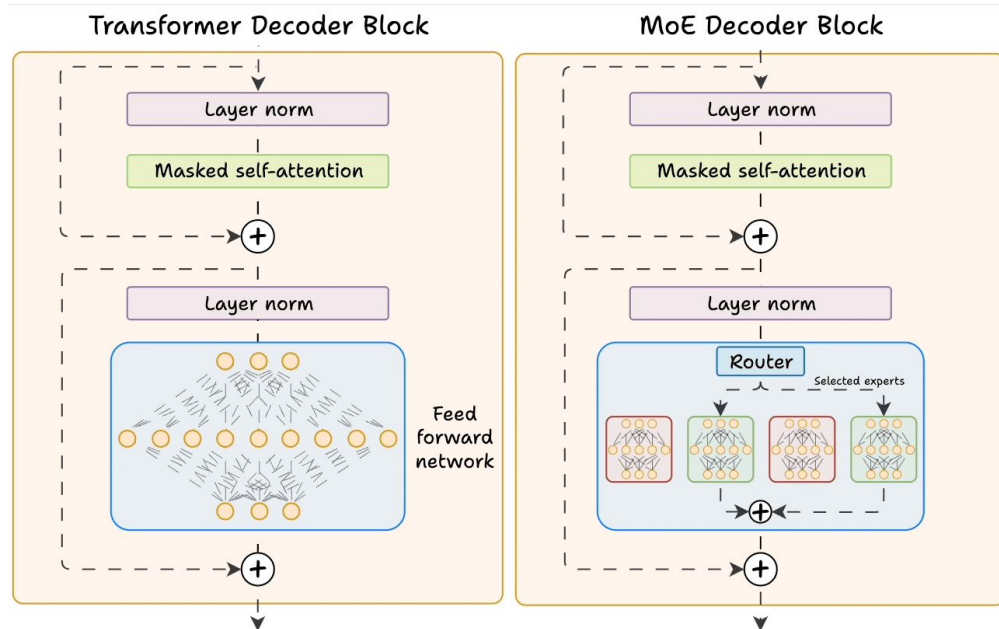
Decoder
Generación

Transformer

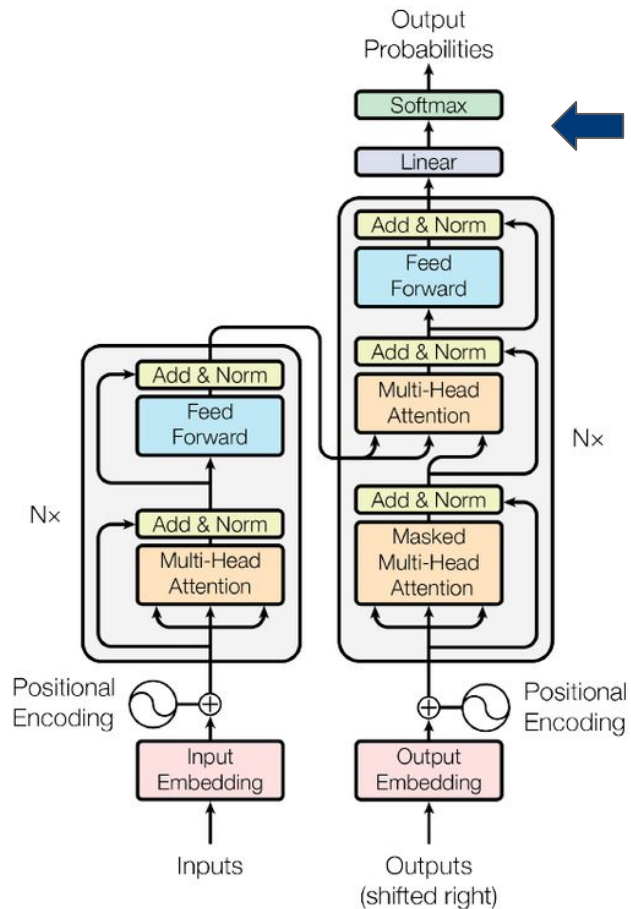
Feed Forward, Mixture of Experts



Un Mixture of Experts es una variante del transformer donde el bloque feedforward se reemplaza por múltiples redes especializadas (expertos), y un mecanismo de enrutamiento decide cuáles usar para cada token.



Transformer



Encoder
Representación

Decoder
Generación

Transformer

Softmax



Which word in our vocabulary
is associated with this index?

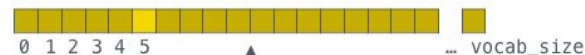
am

Get the index of the cell
with the highest value
(argmax)

5

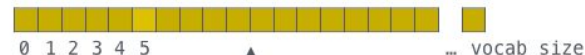
```
for cur_pos in range(min_prompt_len, total_len):
    logits = self.model.forward(tokens[:, prev_pos:cur_pos], prev_pos)
    if temperature > 0:
        probs = torch.softmax(logits[:, -1] / temperature, dim=-1)
        next_token = sample_top_p(probs, top_p)
    else:
        next_token = torch.argmax(logits[:, -1], dim=-1)
```

log_probs



Softmax

logits



Linear



Decoder stack output

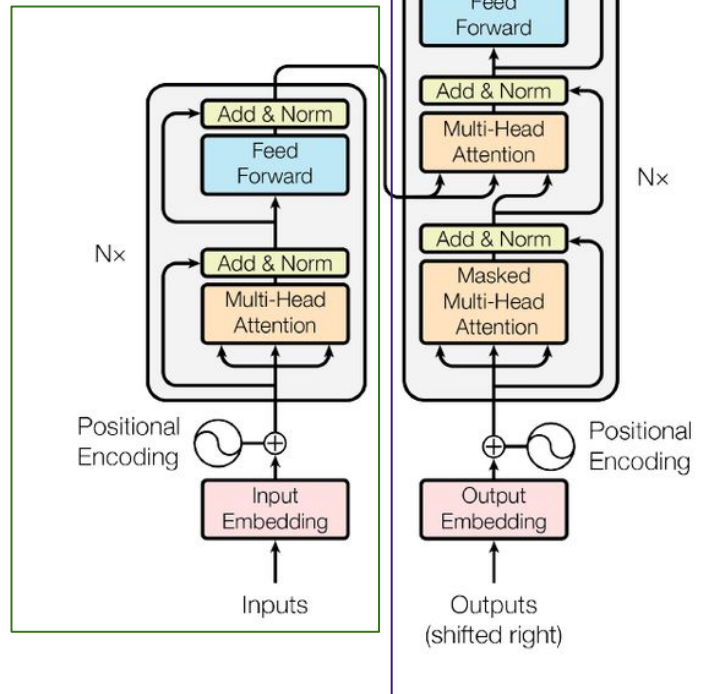
Transformer

BERT



Encoder
Representación

Encoder-Decoder
T5
BART



GPT 

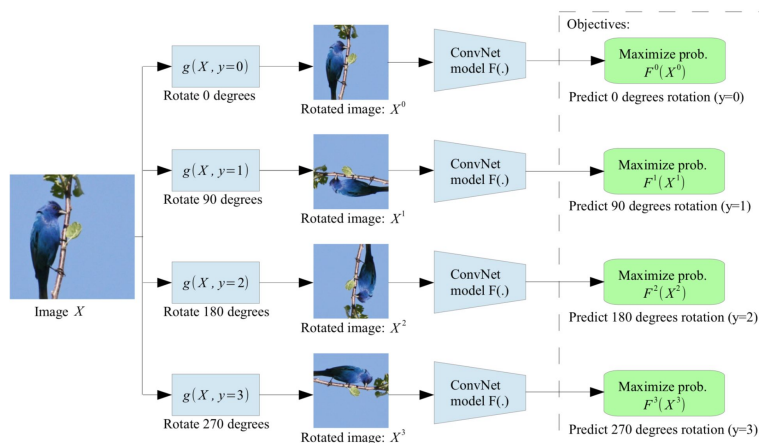
Decoder
Generación

Self-Supervised Learning



Es un tipo de aprendizaje donde el modelo genera sus propias etiquetas a partir de los datos, sin necesidad de anotación humana.

Permite entrenar con cantidades masivas de datos no etiquetados (todo internet, básicamente), y luego hacer fine-tuning con pocos ejemplos etiquetados. Es la base de todos los LLMs modernos.

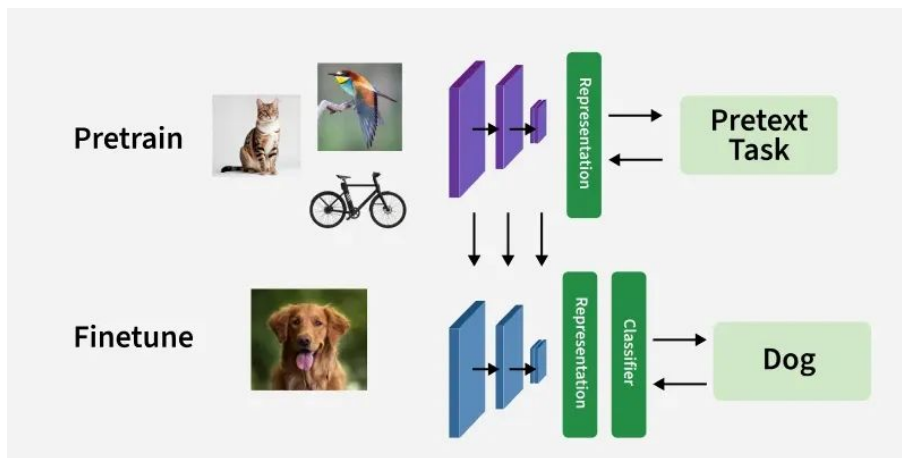


Self-Supervised Learning



Es un tipo de aprendizaje donde el modelo genera sus propias etiquetas a partir de los datos, sin necesidad de anotación humana.

Permite entrenar con cantidades masivas de datos no etiquetados (todo internet, básicamente), y luego hacer fine-tuning con pocos ejemplos etiquetados. Es la base de todos los LLMs modernos.

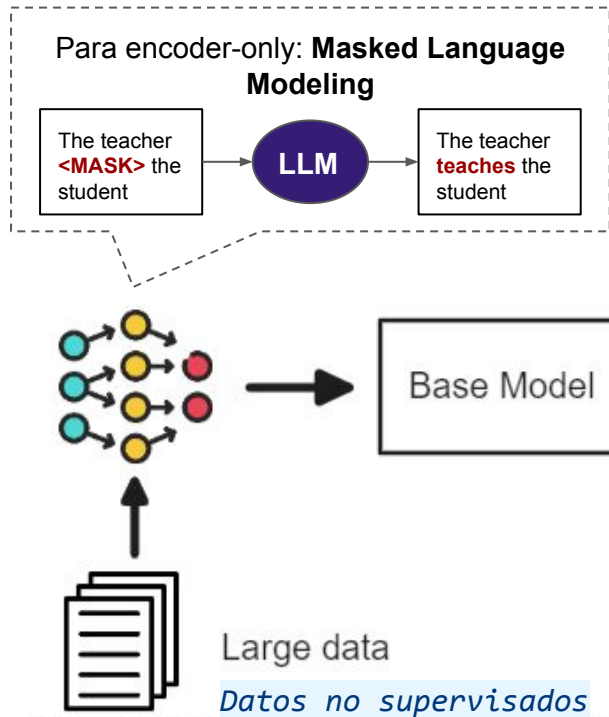


Pre-training

El pretraining es el proceso de **entrenar un modelo de lenguaje desde cero sobre grandes volúmenes de texto no etiquetado**, con el objetivo de aprender representaciones generales del lenguaje mediante tareas auto-supervisadas.



Definimos objetivos para “lidar” con los datos no supervisados

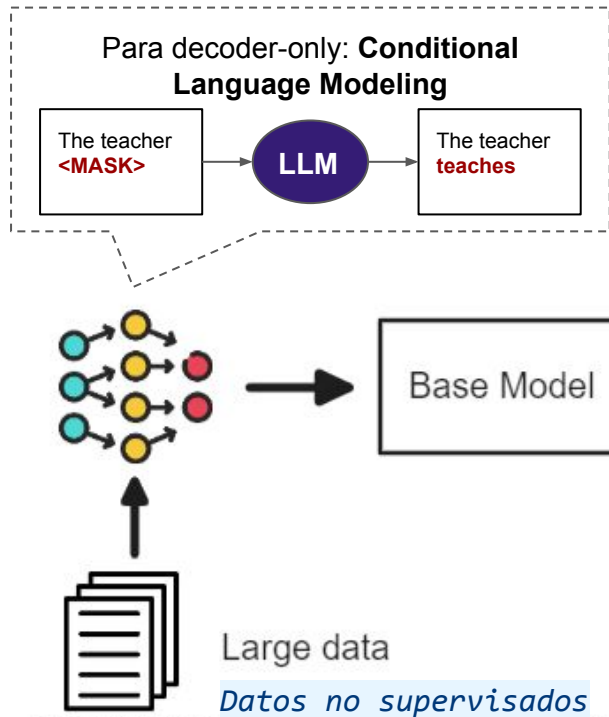


Pre-training

El pretraining es el proceso de **entrenar un modelo de lenguaje desde cero sobre grandes volúmenes de texto no etiquetado**, con el objetivo de aprender representaciones generales del lenguaje mediante tareas auto-supervisadas.



Definimos objetivos para “lidar” con los datos no supervisados



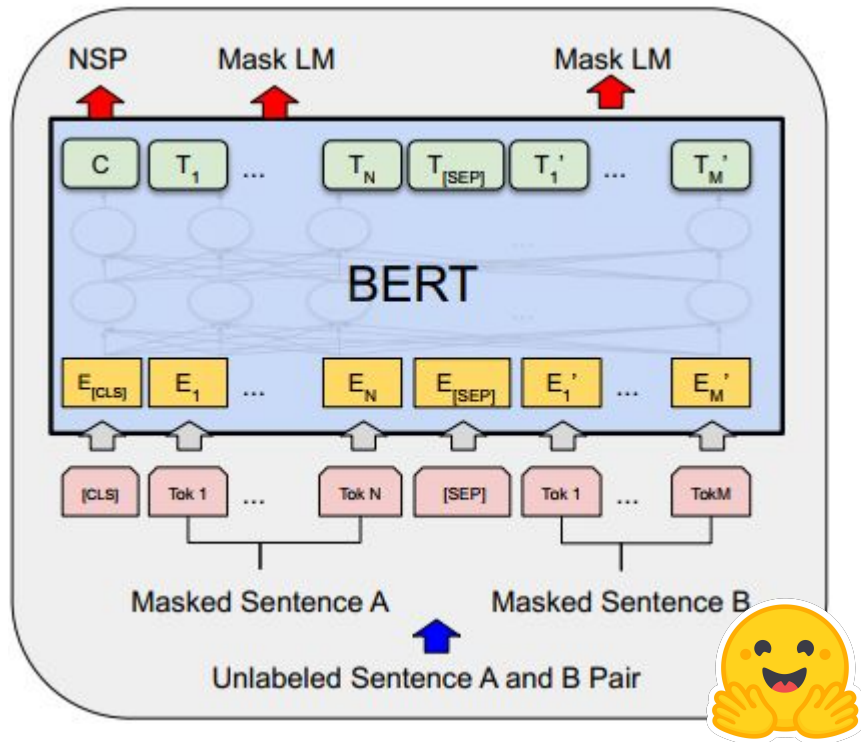
BERT (2019)

“Pre-training of Deep Bidirectional Transformers”

[LINK](#)

[LINK](#)

[LINK](#)



[Creado por Hugging Face/Google](#)

Su arquitectura se basa en stack de layers de transformers (encoders)

NSP: Se entrenó observando dos secuencias de entrada, su misión era determinar si la segunda secuencia estaba relacionada o no con la primera.

MLM: De forma aleatoria en cada inferencia se ocultó una palabra de cada secuencia (masked)

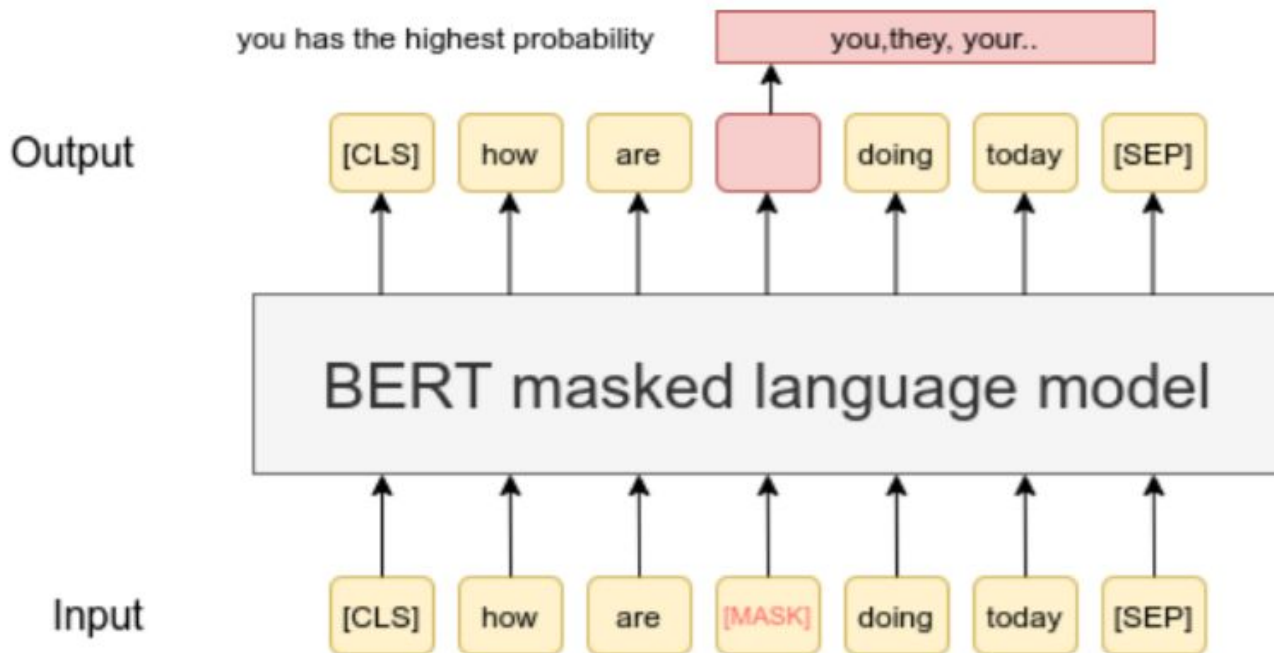
Utiliza la tokenización WordPiece

BERT (2019)

“Pre-training of Deep Bidirectional Transformers”



MLM

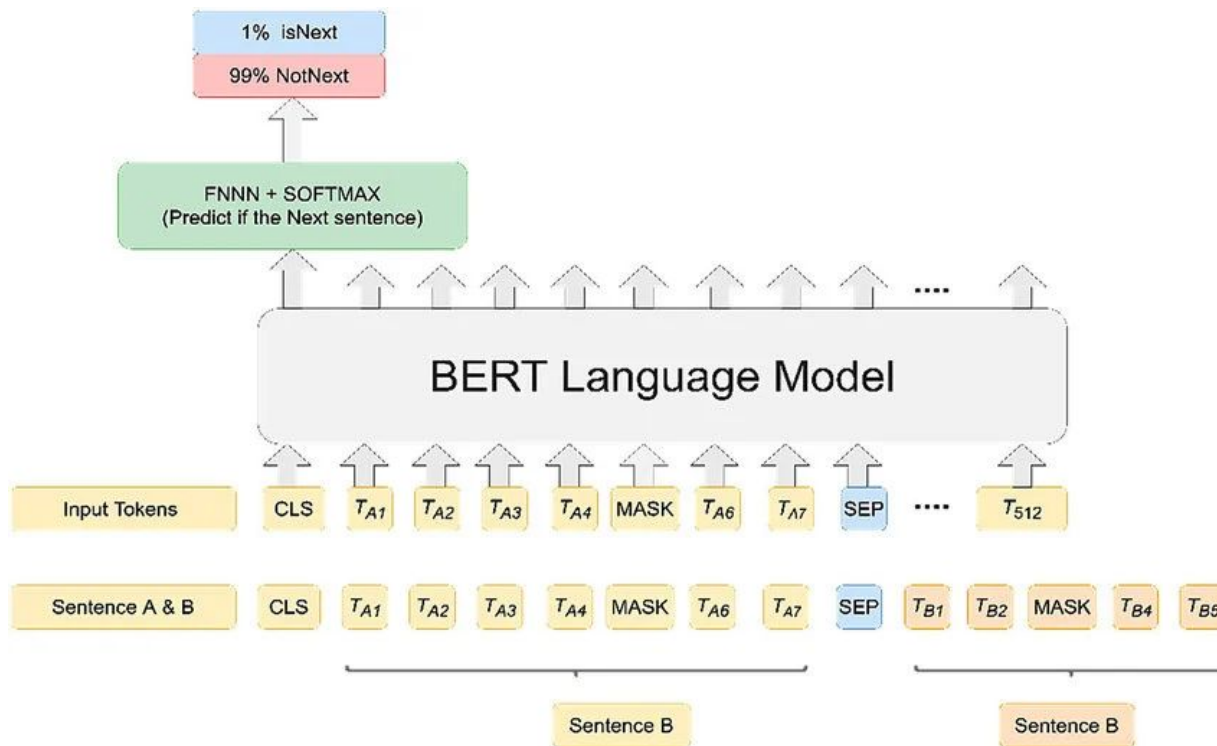


BERT (2019)

“Pre-training of Deep Bidirectional Transformers”



NSP





Link al Colab



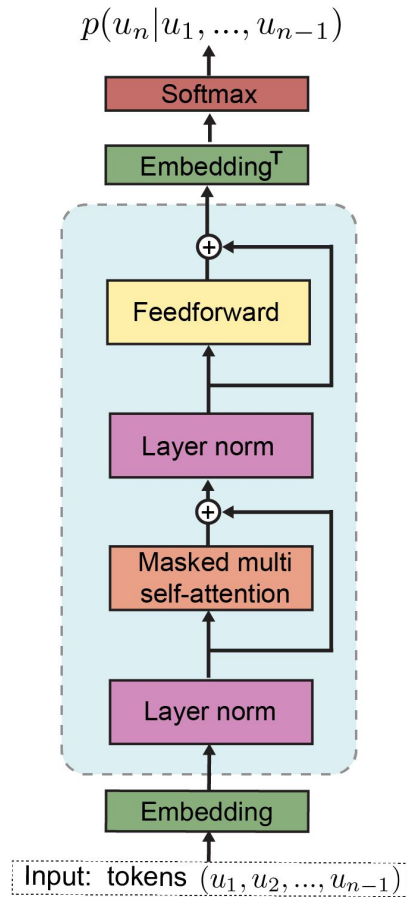
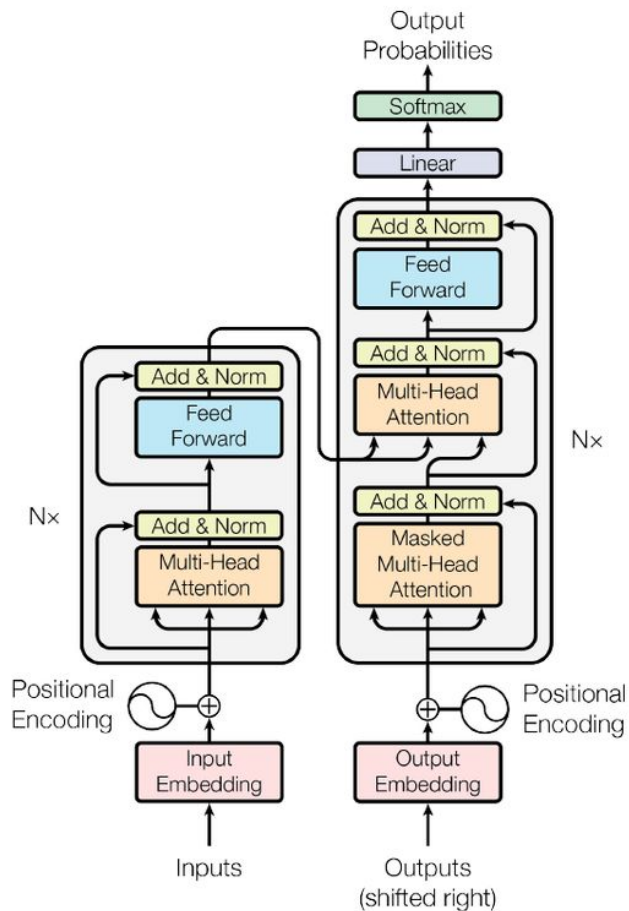
[LINK](#)



Link al Colab



[LINK](#)

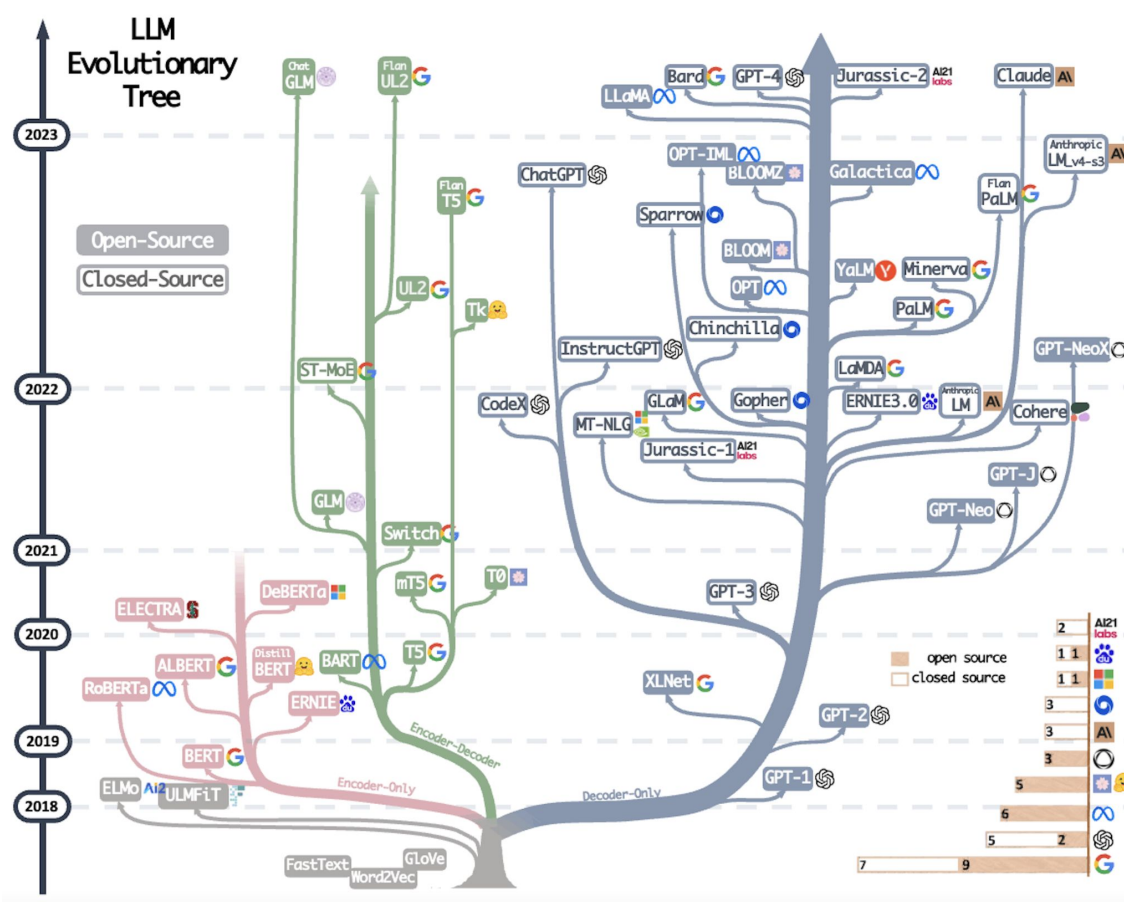




Link al Colab



[LINK](#)





Link al Colab



LINK